SecondChance v21 독해: 몇 가지 규칙의 비교에서 전체 감사 일정의 경계로

이 앞부분은 v21에서 실제로 새로 계산한 질문을 쉬운 말로 설명한다. 뒤의 v20 해설 전체와 기존 부정적 결과는 그대로 보존한다. 새 모델·자연스러운 실패 이력·실제 재감사·독립 표본은 추가되지 않았다. 새로 한 일은 이미 있는 회복 경로에서 가능한 감사 일정을 빠짐없이 비교한 정확한 계산이다.

v21 독해 1. 이전에 답하지 못했던 질문

평가기의 오판을 고쳤다고 해서 현재 에이전트가 고쳐진 것은 아니다. 예전에 버린 답은 맞을 수 있지만 현재 선택된 프로그램, 남은 교훈, 이후의 정당한 변경이 여전히 잘못되어 있을 수 있다. SecondChance는 그래서 저장 답을 다시 채점하는 일, 후보를 새로 실행하는 일, 통합된 현재 상태를 감사하는 일을 구분한다.

감사도 무제한으로 할 수 없다면 언제 어떤 상태를 확인해야 할까? 이전 판본은 FIFO, 늦게 조회, 제한된 미래 모형을 쓰는 규칙 등 아홉 규칙을 비교했다. 그중 한 규칙이 마지막에는 더 정확한 폭을 남기지만, 중간까지의 불확실성은 더 크게 유지하는 경우가 있었다. 그러나 이것만으로 마지막 정확도와 중간 불확실성이 반드시 충돌한다고 말할 수는 없다. 비교한 규칙 목록 밖에 둘을 함께 개선하는 일정이 있을 수 있기 때문이다.

이번 질문은 바로 이것이다. 회복 경로는 바꾸지 않고, 허용된 감사 일정을 전부 놓고 보면 기존 규칙들은 어디에 있는가? 이 질문의 답은 현재 비교의 빈틈을 보여 준다. 미래를 모르는 현장 정책을 새로 개발했다는 뜻은 아니다.

v21 독해 2. 모든 스케줄을 비교한다는 뜻

일정의 규칙은 단순하다. 회복 시간이 0일 때에는 감사하지 않는다. 이후 정수 체크포인트마다 이미 나타난 상태 하나를 처음 감사하거나 그냥 넘어간다. 한 시점에 두 상태를 감사할 수 없고, 같은 상태는 캐시가 있으므로 다시 비용을 내지 않는다. 전체 조회 횟수는 정한 상한을 넘을 수 없다. 아직 나타나지 않은 상태를 미리 감사하는 일정도 금지한다.

여기서 중요한 가정이 있다. 감사한 순간 저장된 세 평가 패널의 값이 즉시 공개되며 회복 시계는 멈추거나 늘어나지 않는다. 실제 검사 시간이 측정됐다는 뜻은 아니다. 미래에 실제 감사 지연이나 실패가 생기면 같은 일정 계산을 그대로 사용할 수 없다. 또 여러 이력에 같은 상태 SHA가 나타나면 같은 고정 입력 효용을 공유한다고 가정한다. 이 가정은 작성된 결정적 자료에서만 확인됐고 실제 에이전트의 숨은 상태 완전성까지 인증하지 않는다.

계산은 매 순간 '현재까지 확인한 상태 목록'별로 가장 작은 누적 불확실성을 보관한다. 같은 시간에 같은 상태 목록을 얻었다면 앞으로 선택할 수 있는 감사와 마지막 폭은 같다. 그중 지나온 불확실성만 더 큰 일정은 더 작은 일정으로 바꿔도 손해가 없다. 이것이 동적계획법으로 중복 계산을 줄일 수 있는 이유다. 별도 검산기는 이 논리를 재사용하지 않고 허용된 일정을 실제로 하나씩 전부 세어 같은 답인지 확인했다.

정답 점수를 미리 본 것 아니냐는 질문도 구분해야 한다. 이번 폭 계산은 아직 모르는 상태의 계수가 얼마나 남았는지에만 의존한다. 따라서 점수 내용을 지워도 최적화 결과가 같다. 하지만 점수를 안 본다는 것과 미래를 안 본다는 것은 다르다. 이 사후 분석자는 앞으로 어떤 상태가 나타나고 언제 사라질지 전체 경로를 안다. 온라인 규칙이 가질 수 없는 이 권한을 가진 기준을 구조 oracle이라고 부른다.

v21 독해 3. 최선 경계가 점 하나라는 결과

마지막 안전폭은 회복이 끝났을 때 남는 모름의 크기다. 면적은 도중에 얼마나 오래, 얼마나 크게 모르는 상태였는지를 정해진 체크포인트로 더한 것이다. 둘 다 작을수록 좋다. 마지막 폭이 작다고 도중 면적도 작아지는 것은 아니므로 처음에는 두 목적의 최선 경계를 계산한다.

회복 상한	조회 상한	최선의 마지막 폭	최선의 면적
5	1	1/3	31/25
5	2	1/5	6/5
5	3	2/15	6/5

5	4	1/15	6/5
8	1	1/8	75/64
8	2	1/12	37/32
8	3	1/24	73/64
8	4	0	217/192

이번 여덟 계약에서는 각각 하나의 점이 두 목적을 동시에 최소화했다. 일정 자체가 하나라는 뜻은 아니다. 같은 점을 만드는 일정은 여러 개 있고 모두 동률로 기록했다. 모든 다른 자료에서도 최선 경계가 점 하나라는 일반 정리 역시 아니다. 실제 계산한 두 고정 회복 경로에서 얻은 결과다.
기존 정책 설정 72개 중 28개는 그 경계에 닿고 44개는 두 목적상 다른 허용 일정에 지배된다. 회복 상한 8, 조회 상한 3 또는 4에서는 기존 아홉 규칙 중 경계에 닿는 규칙이 없다. 반대로 일부 상한 5 규칙과 상한 8의 적은 조회 조건에서 FIFO가 동률인 결과도 그대로 남겼다. 이것을 새 규칙의 보편적 우월성으로 바꿔 말하지 않는다.

v21 독해 4. B8 세 번 조회를 직접 계산하기

상한 8에서 세 번 조회할 때 구조 oracle의 예시는 시점 1·2·4에서 감사한다. 그 상태들은 각 시점에 실제로 도착해 있으므로 기회 규칙을 어기지는 않는다. 그러나 시점 4에 나타난 여러 상태 중 어떤 것을 골라야 마지막까지 유리한지는 전체 미래 경로를 아는 계산으로 정했다. 현장에서 이를 예측할 수 있다는 근거는 없다.
시점 0부터 8까지 이 예시의 안전폭은 다음과 같다. 다음 목록은 교육용 가상 숫자가 아니라 기존 자료를 사용한 v21의 정확한 계산이다.

$$2,\ \frac{7}{4},\ \frac{3}{2},\ \frac{5}{4},\ 1,\ \frac{19}{24},\ \frac{13}{24},\ \frac{7}{24},\ \frac{1}{24}$$

면적은 시점 0부터 7까지만 더해 8로 나눈다. 마지막 시점 8의 폭은 그 뒤에 머무르는 시간이 없으므로 면적에 다시 넣지 않는다. 앞의 다섯 폭 합은 15/2, 뒤의 세 폭 합은 13/8이다.

$$J = \frac{15/2 + 13/8}{8} = \frac{73}{64}$$

이전 FIFO와 두 union 규칙은 마지막 폭 1/12, 면적 37/32였다. 새 구조 기준의 마지막 폭은 1/24여서 폭 차이는 1/24다. 면적 차이는 다음과 같다.

$$\frac{37}{32} - \frac{73}{64} = \frac{1}{64}$$

늦게 세 번 읽는 규칙은 마지막 폭 1/24로 동률이지만 면적은 75/64라서 구조 기준보다 1/32 크다. 따라서 이전 규칙 사이의 '끝은 좋아졌지만 도중은 나빠졌다'는 관측은 맞아도, 같은 회복 경로에서 두 목적이 반드시 충돌한다는 결론은 나오지 않는다.
폭만 최적화한 계산이므로 부호 인증 시점까지 최적화한 것은 아니다. 실제 정책의 미도달과 기존 unaffected 인증 시점은 이전 기록 그대로다. 또 이전 v20 사례에서 시점 6의 안전폭은 7/12, 시점 8의 마지막 폭은 1/12였다는 사실과 이번 다른 일정의 곡선을 섞지 않아야 한다.

v21 독해 5. 분모와 검산을 혼동하지 않기

독립 전수 검산은 33,658개의 스케줄-계약 조합을 확인했다. 조회 상한 1에서 가능한 일정은 상한 2·3·4에서도 다시 포함되므로 이 숫자에는 반복이 있다. 최대 조회 상한만 보았을 때 고유한 회복상한·일정 쌍은 1,175+24,729=25,904개다. 어느 쪽도 자연 이력이나 독립 통계 표본 수가 아니다. 여전히 열두 authored template에서 파생된 계산이다.
코드와 계획, 172개 입력의 해시를 새 실행에 고정했다. 다른 구현은 각 단위 시간 구간의 상태 점유를 다시 세고 모든 일정을 열거했다. 최선 점, 동률 일정 수, 상태 집합별 최소 면적과 기존 72개 정책의 수치가 일치했다. 아직 도착하지 않은 상태 조회, 같은 상태 재조회, 시점 0 조회, 한 시점의 두 조회 등 잘못된 일정 14개도 거절했다.

이것은 산술과 구현을 다른 방식으로 확인한 것이다. 같은 저자가 같은 자료로 만든 검산기이므로 외부 경험적 재현이나 실제 에이전트 복구 효과 인증은 아니다. SHA 일치와 좋은 모델 비평도 새로운 자연 이력을 만들어 주지 않는다.

v21 독해 6. 실제 에이전트 자료로 옮기려면

별도 읽기 자료는 세 가지다. AgentDebug/AgentErrorBench는 오류 궤적과 재실행을, Who&When은 어느 에이전트의 어느 단계가 실패를 유발했는지의 주석을, AgentDebugX는 기록·진단·복구 분기를 검토하는 데 연결된다. 원문을 읽은 범위와 접근·라이선스 제약은 literature/RESOURCES.json에 있다. 자료나 코드를 설치·실행하지 않았다.

우리에게 필요한 것은 오류 위치 정답만이 아니다. 같은 task 입력과 프로그램·교훈·메모리·도구·환경을 정말 같은 상태로 복원할 수 있는지, 동일 상태를 다시 감사하면 결과가 안정적인지 확인해야 한다. 그다음에 수정 전후의 현재 상태와 유효한 나중 변경의 보존을 따로 평가해야 한다. 대화 로그가 있다는 이유만으로 이 조건들이 자동 충족되지는 않는다.

다음 자연 이력 실험에서는 미래 계수를 보기 전에 규칙을 고정하고, 실제 감사의 호출·토큰·실패·경과 시간을 함께 기록해야 한다. 스케줄을 미래까지 보고 고른 이번 정답표를 정책으로 바꿔 같은 노출 자료에서 다시 시험하는 것은 독립 검증이 아니다.

다음 여섯 질문으로 이해를 점검하자. ① 정답 점수를 보지 않는 구조 oracle은 왜 여전히 미래 특권을 가지는가? ② 같은 시간과 상태 목록에서 누적 면적이 큰 경로를 버려도 되는 이유는? ③ 1/64와 1/32의 차이는 각각 누구와의 비교인가? ④ 33,658은 왜 고유 일정 수가 아닌가? ⑤ 시점 8의 조회가 마지막 폭은 줄여도 면적은 줄이지 못하는 이유는? ⑥ 실제 지연이 회복 경로를 바꾸면 어떤 계약을 다시 세워야 하는가?

이제부터는 기존 v20 해설 전체다. 기존의 세 가지 폭, tube의 정확한 가정, 1,620개 실제 premium 0, 열세 종속 view와 최종 폭 동률을 삭제하지 않았다.

SecondChance v20 독해 길잡이: 무엇이 식별되며 무엇은 아직 모르는가

이 길잡이는 짧은 본문을 읽고 상세 보충자료로 넘어가기 위한 새 앞부분이다. 아래 보존본은 삭제하거나 고쳐 쓰지 않았다. v20은 새로운 모델 실험 결과가 아니라 같은 v19 근거를 압축·확장해 읽을 수 있게 한 판본이다. 논문을 짧게 만들었다고 불리했던 대조군이나 불확실성까지 짧게 지워서는 안 된다. 그래서 본문에는 실제 premium 0, 마지막 폭의 16개 동률, 작은 면적 개선 네 건, 실제 감사·모델 호출 0을 남겼다.

1. 이 연구가 해결하는 질문과 해결하지 못한 질문

먼저 "옛날 채점을 고쳤다"와 "현재 프로그램이 고쳐졌다"를 구분하자. 저장된 답의 점수가 바뀌어도 현재 코드·나중에 적용한 정상 수정·남아 있는 잘못된 교훈은 그대로일 수 있다. E1은 옛 답의 재채점, E2는 후보의 새 실행, E3는 통합된 현재 상태의 새 감사다. 어느 칸의 성공인지 말하지 않고 "회복했다"고 하면 연구 질문이 바뀐다.

이번 중심 질문은 E3를 확인할 정보가 부족할 때, 어떤 상태를 먼저 확인해야 정책 비교의 불확실성이 줄어드는가다. 현재까지의 경로만 보는 선택기는 미래 commit을 알 수 없다. 더 복잡한 최적화 목적을 썼다는 사실과 최종 비교가 더 정확해졌다는 사실도 다르다. 새 규칙의 최종 폭은 대응 대조군과 16건 모두 같았다. 따라서 이 논문을 "새 에이전트가 기존 방법보다 회복을 잘한다"로 발표하면 안 된다.

2. 본문·보충자료·해설을 오가는 독해 지도

본문은 질문과 핵심 결과를 판단하는 입구다. 증명이나 모든 조건을 외우려고 처음부터 역사 원고를 통독할 필요는 없다. 아래 질문 중 막힌 곳만 골라 해당 보충 절로 가고, 마지막에 보존 원고로 구현·개발 이력을 확인하자. S 번호는 새 SUPPLEMENT.json의 정확한 절 번호다.

본문 절	새 보충 절	스스로 던질 쉬운 질문
Introduction	S1, S14	재채점 성공과 현재 상태 성공은 왜 다른가?
Related Work	S8, S13	기존 self-correction 방법과 정보·목표가 같은가?
Problem and Method	S2–S7	어떤 것을 알고 난 뒤의 폭이며 무엇을 모르는가?
Experimental Setup	S8, S12	선택기는 정확히 무엇을 볼 수 있고 비용은 무엇인가?
Results	S9, S10, S11	1/192는 어디서 나오며 16개 비교를 모두 봤는가?
Limitations	S6, S12, S13	상태 동일성·미래 모형이 틀리면 무엇을 중단하는가?
Conclusion	S14	증명, 관측, 제안 중 무엇을 실제로 얻었는가?

3. 손계산 1: 모든 상태를 알아도 남는 시나리오 불확실성

두 상태의 효용을 이미 알고 있고 값이 (1,0)이라고 하자. 미래 시나리오 0의 계수는 (1/4,−1/4), 시나리오 1은 (−1/4,1/4)다. 첫 시나리오의 대비값은 1/4, 둘째는 −1/4다. 어느 시나리오인지 알려준 뒤에는 미지 효용이 없으므로 각 시나리오 안의 폭은 0이다. 그러나 시나리오 자체가 미정이면 가능한 대비값의 전체 범위가 남는다.

$$W_{\mathrm{cond}} = 0, \qquad W_{\mathrm{union}} = 1/4 - (-1/4) = 1/2.$$

이것이 첫 두 폭의 차이다. 세 번째 F는 아직 공개되지 않았다고 취급하는 효용값까지 최악으로 바꿔 보는 계획 목적이다. 의도적으로 효용을 보지 않는 선택 규칙이므로 이미 조회한 값도 선택기에는 넘기지 않고, 조회한 상태의 이름만 남긴다. 실제 공개 효용을 고정한 두 번째 폭과 F를 같은 숫자로 부르면 안 된다.

이 예는 수학적으로 가능한 구성을 보여준다. 실제 저장 패널 1,620건에서는 두 번째 폭과 첫 번째 폭의 차이가 전부 0이었다. 반례가 있으니 실제 양의 사례가 있었다고 바꿔 쓰면 안 된다. "가능한 이유를 증명했다"와 "그 현상을 관측했다"를 먼저 분리하는 연습이다. S3–S5에서 구간의 끝점과 네 endpoint 조합을 유도해 보자.

4. 선택기가 볼 수 있는 정보부터 확인하기

공개 정보는 horizon B, 현재 checkpoint t, 이미 도착한 상태의 이름·시각, 완료된 prefix의 계수, 현재 arm의 상태 빈도 차이, 지금까지 조회한 이름이다. 미래 commit, 최종 계수, 코드 의미, 효용 payload는 선택기 입력이 아니다. 감사값은 선택이 끝난 다음 별도 구간 계산기에 들어간다. 결과를 알고 만든 hindsight oracle은 이 아홉 causal 규칙과 동급 경쟁자로 세지 않는다.

set 규칙은 현재 보이는 집합 중 남은 조회 한도와 기회에 맞는 계획을 계산하고 맨 앞 하나만 지금 실행한다. 다음 시점에 다시 계획한다. 미래에 나타날 상태를 미리 조회하는 규칙도, 모든 온라인 정책 중 전역 최적을 증명한 규칙도 아니다. 숨긴 미래와 효용을 바꾸되 공개 prefix를 유지했을 때 선택이 같아야 한다는 것이 중요한 검산이다.

새 zero-sum L1 tube의 정확한 폭 증가도 조건부다. 새로 등장할 두 상태의 효용을 0과 1로 둘 수 있고, 그쪽으로 양·음의 질량을 옮길 수 있어야 epsilon만큼의 폭 증가가 달성된다. 이 조건을 빼고 물리적으로 가능한 미래에서도 항상 같다고 말할 수 없다. 같은 eta와 r로 모든 후보에 같은 epsilon을 더하면 순위도 바뀌지 않는다. 포함률이 좋아지는 것과 선택이 좋아지는 것이 다른 이유다.

5. 손계산 2: checkpoint 6과 마지막 checkpoint를 바꾸지 않기

B=8, q=3의 union-one 경로에서 면적에 들어가는 여덟 safe 폭은 순서대로 2, 7/4, 3/2, 5/4, 1, 5/6, 7/12, 1/3이다. t=6의 7/12는 중간 시점 폭이다. t=8의 1/12는 마지막 폭이다. "t=6에 이미 1/12"라고 읽으면 앞으로 남은 불확실성을 지워 버린다. 표 S9의 행을 실제 checkpoint와 맞춰 읽자.

$$\sum_{t=0}^{7} W_t = 37/4, \qquad J = (37/4)/8 = 37/32.$$

면적은 t=0부터 7까지 합하고 horizon 8로 나눈다. t=8은 다음 양의 길이 구간이 없어서 면적 항이 아니다. 조회 횟수 3으로 나누지도 않는다. 제한 one 규칙은 t=5에서만 7/8이고 새 규칙은 5/6이므로 차이가 1/24다. 한 칸의 차이를 horizon으로 나누면 정확히 1/192 개선이다. 나머지 시점과 최종 폭은 같다. 같은 시각에 같은 수만큼 저장값을 봤으므로 실제 감사 실행이 빨라졌다는 계산도 아니다.

6. 손계산 3: 마지막 폭이 좋은 정책이 면적에서도 좋은가

late-prefix는 t=6,7,8에 조회해 마지막 폭 1/24를 얻는다. union 규칙의 1/12보다 작다. 하지만 late 면적은 75/64이고 union 면적은 37/32다. 분모를 맞춰 보자.

$$75/64 - 37/32 = 75/64 - 74/64 = 1/64.$$

즉 마지막 정밀도는 좋아졌지만 그전까지 불확실성을 유지한 누적 부담은 더 크다. 마지막 양의 unaffected certificate도 t=8에서야 나온다. "더 좋은 정책"이라는 말 앞에 마지막 폭, 면적, 증명 시각, 조회 수 중 어느 축인지 붙여야 한다. 표 S10에는 새 one/set와 기존 one/set의 16개 계약을 전부 남겼다. 4개 면적 감소만 발췌하지 말고 나머지 12개 동률과 최종 16개 동률을 같이 확인하자.

7. 13개 view와 eta를 읽을 때의 금지된 지름길

B5의 t=0..5와 B8의 t=0..8을 합하면 15개 view다. 마지막 두 개는 남은 미래가 0이라 포함이 자명하므로 빼서 13개다. 이 13개에도 조회가 금지된 t=0 두 개가 들어 있다. 실제 preterminal 조회 시점만 보면 11개라는 다른 분모가 된다. 분모를 바꾸고 같은 비율인 것처럼 읽으면 안 된다.
eta를 0에서 1/2로 키우면 같은 13개 중 포함 수가 4에서 13으로 늘어난다. 새로운 13개 표본이 생긴 것이 아니며 coverage 100%가 검증된 것도 아니다. 최종 계수를 본 뒤 eta를 골랐다면 미래에 쓸 독립 calibration은 아니다. 특히 "보이는 상태 네 개를 이미 조회한 집합"과 "과거 조회 기회로 실제 만들 수 있었던 집합"은 다르다. 불가능했던 계획 집합의 최대 premium을 실제 정책 결과로 옮기지 말자.

8. 스스로 설명하는 확인 문제와 다음 읽기

1. 재채점 12/12와 통합 상태 0/12가 모순인가? 아니다. 같은 답·후보·현재 상태 중 채점 대상이 다르다. S1에서 E1/E2/E3를 자기 말로 설명하자. 2. W_cond=0이면 최종 대비를 아는가? 아니다. 위 두 상태 예에서 시나리오 미정이면 union 폭은 1/2다. S3의 알려진 기여가 시나리오에 따라 이동하는 점을 짚자. 3. J 계산에 W8=1/12를 더해야 하는가? 아니다. J는 [0,8)의 왼쪽 계단 합이다. 마지막 폭은 별도 결과다. S9의 여덟 항을 다시 더하자. 4. eta를 크게 해서 포함이 늘면 정책 순위도 개선되는가? 이 uniform tube에서는 아니다. 같은 상수가 모든 후보에 더해진다. S6의 달성 조건을 빼지 말자. 5. 네 번의 1/192 개선으로 새 모델 우위를 주장할 수 있는가? 없다. 종속된 저장 경로의 기술적 계산이고 새 모델·실제 감사는 0이다. S10의 모든 동률도 답에 포함하자. 6. 물리적 제약 때문에 fresh 상태의 효용 0·1이 불가능하면 무엇을 고쳐야 하나? 정확히 epsilon이라는 달성 주장을 다시 검토해야 한다. 정당한 상한과 실제 달성은 다르다. S6, S13을 읽자.
첫 독해 뒤에는 본문의 세 폭을 외워 설명하고, S4의 네 조합을 손으로 유도하고, S9의 1/192를 계산해 보자. 그 다음에만 아래 역사 원고를 펼쳐 과거 실패·대조군·조판 이력이 어떤 경계 조건을 만들었는지 확인하자. 지금 얻지 못한 자연 회복 효과, 실제 감사 지연, 독립 검증 표본을 모두 얻었다고 느끼지 않는 것이 이 독해의 마지막 확인 기준이다.

보존된 v19 해설 원문

SecondChance v19: 시나리오를 안다는 조건과 모른다는 조건

이 문서는 연구 분석의 해설·원고 통합을 위한 자료다. 노출된 authored 경로 168개·12개 템플릿·9개 상태를 재사용했다. 새로운 실제 감사·자연 이력·독립 표본·모델 호출은 모두 0이다. 자연 에이전트의 회복 효과나 실제 감사 속도를 높였다는 결론은 없다.

서로 다른 세 폭

현재 prefix 점유 계수를 p, 남은 비용 비율을 r, 현재 두 arm의 상태 빈도 차이를 d라고 하자. 기존 제한 미래 모형은

$$c(\lambda) = p + \lambda r d, \quad 0 \le \lambda \le 1, \qquad c_0 = p,\ c_1 = p + rd$$

다. 이미 조회한 상태 집합은 A, 상태 효용은 $u_s \in [0,1]$이다.
1. 미래 시나리오를 알고 난 뒤 남는 미지 효용의 폭 중 최악을 묻는 질문. 2. 이미 조회한 실제 효용은 고정하지만 미래 시나리오도 모를 때 가능한 대비값 전체를 묻는 질문. 3. 조회 집합의 효용을 아직 보기 전, 어떤 결과가 오더라도 확보할 수 있는 최악의 식별 폭을 묻는 질문.
첫 질문의 답은 다음과 같다.

$$W_{\mathrm{cond}}(A) = \max_{i \in \{0,1\}} \sum_{s \notin A} |c_{i,s}|.$$

절댓값 합은 λ의 볼록함수여서 endpoint에서 최댓값이 나온다. 그러나 이 값은 시나리오를 안다는 조건 아래의 폭이다.
고정된 u_A에서 각 endpoint의 구간은

$$L_i = \sum_{s \in A} c_{i,s} u_s + \sum_{s \notin A} \min(c_{i,s}, 0), \quad U_i = \sum_{s \in A} c_{i,s} u_s + \sum_{s \notin A} \max(c_{i,s}, 0).$$

실제 미지 시나리오 구간은 $[\min_i L_i, \max_i U_i]$이다. lower endpoint는 λ에 오목하고 upper endpoint는 볼록하므로 각각의 극값도 endpoint에서 나온다. λ와 미지 효용의 곱공간은 연결되어 있어 사이의 값도 채워진다. 이 폭은 각 시나리오의 폭만 최대한 값과 일반적으로 다르다.
마지막 질문의 utility-blind 사전 최악 폭은 네 endpoint 교차 조합으로 정확히 계산한다.

$$F(A) = \max_{i,j \in \{0,1\}} \left[\sum_{s \in A} \max(c_{i,s} - c_{j,s}, 0) + \sum_{s \notin A} \left\{ \max(c_{i,s}, 0) - \min(c_{j,s}, 0) \right\} \right].$$

유도는 $U_i - L_j$를 전개하고, 아직 모르는 u_A에 대해 선형항을 최대화하는 것이다. 양의 계수에는 1, 음의 계수에는 0을 넣는다. 따라서 실제 패널의 효용을 선택기에 주지 않고 계산할 수 있다. 실제 패널의 union 폭과 이 사전 최악을 같은 값처럼 부르면 안 된다.

직접 만든 두 상태 반례

이는 새로운 자연 이력이나 benchmark sample이 아닌 수학적 구성이다. $p = (1/4, -1/4)$, $c_1 = (-1/4, 1/4)$이고 두 효용이 이미 $(1,0)$으로 알려졌다고 하자. 시나리오 0에서는 대비가 $1/4$, 시나리오 1에서는 $-1/4$다. 시나리오를 알면 폭은 각각 0이지만 λ를 모르면 $[-1/4, 1/4]$, 폭 $1/2$가 남는다. 원래 상태를 전부 감사해도 미래 점유 계수의 불확실성은 남는다.

중요한 음성 결과는 실제 저장된 세 패널의 1,620개 정책·시점 조합에서는 추가 union 폭이 전부 0이라는 점이다. 반복된 정책·q·checkpoint는 독립 관측 1,620개가 아니다. 양의 폭 분리는 반례와 반사실적 효용 검산으로만 보이며 실제 발생률로 보고할 수 없다.

일률적인 L1 tube는 왜 선택을 바꾸지 못하는가

$$\mathcal{T}_\epsilon = \left\{ c(\lambda) + \delta : \sum_s \delta_s = 0,\ \|\delta\|_1 \le \epsilon \right\}, \qquad \epsilon = 2\eta r.$$

여기서는 효용을 0과 1로 둘 수 있는 fresh 좌표 두 개를 허용하며, 더 좁은 실제 실현 가능 미래 집합과 교차하지 않는다. 이 조건을 빼면 아래 등식을 그대로 가져올 수 없다.

zero-sum이라 양·음 질량은 각각 최대 $\epsilon/2$다. 효용 범위 [0,1] 때문에 $|\delta^\top u| \le \epsilon/2$다. fresh 좌표에 $(+\epsilon/2, -\epsilon/2)$를 두고 효용을 (1,0) 또는 (0,1)로 두면 두 극값을 달성한다. 따라서 union 구간은 양쪽으로 $\epsilon/2$씩, 폭은 ϵ 늘어난다. 양쪽에 ϵ을 더해 폭을 2ϵ 늘리는 것은 이 조건에서는 잘못이다.

동일하게 conditioned 폭과 $F(A)$에도 ϵ이 더해진다. 같은 시점 모든 후보에 같은 상수가 붙으므로 고정 η는 순위를 바꿀 수 없다. 포함률 개선과 선택 규칙 개선은 다르다는 명시적인 한계다. 방향별 오차나 support 제한을 바꾸면 이 상수 구조도 바뀔 수 있어 보편적 불가능성은 아니다.

두 상태 반례에 $\epsilon = 1/5$를 더하면 conditioned 폭은 $1/5$, union 구간은 $[-7/20, 7/20]$, 폭은 $7/10$이다. 이것도 교육용 구성이다.

η 포함 민감도는 통계 coverage가 아니다

미리 고정한 η=0,1/8,1/4,1/2,3/4,1에서 같은 비종료 view 13개 중 4,8,10,13,13,13개가 포함됐다. 두 horizon의 반복 prefix이며 이미 노출된 개발 자료다. coverage 100%, 최적 η=1/2, 독립 모형 검증 성공이라고 쓰면 안 된다.

최소 L1 거리는 endpoint 또는 residual이 0인 knot로 구했다. 별도 검산은 같은 loop 대신 가중 중앙값으로 최소점을 계산한다. 실제 최종 계수를 쓴 거리는 사후 평가에만 있고 selector 입력에는 없다.

η=1의 포함은 표본 기반 확률 결론도 아니다. $\lambda = 0$에서 실제 남은 arm 점유 차이의 L1 norm은 최대 $2r$이므로 큰 tube가 포함한다. 이 tube에는 불가능한 미래도 들어갈 수 있다.

새 정책과 최종 동률

one은 현재 단일 조회 중 F를 최소화한다. set은 남은 q와 공개 기회 안에서 현재 보이는 집합을 열거하고 최적 집합의 첫 상태를 하나 조회한 뒤 재계획한다. 엄격한 개선이 없으면 건너뛴다. 선택기에는 효용·미래 commit·final 계수가 들어가지 않는다.

새 2규칙 × 8계약의 최종 unrestricted safe 폭은 대응하는 기존 minimax와 16개 전부 동률이다. B8/q2 one·set, B8/q3 one, B8/q4 one의 네 면적은 정확히 $1/192$ 작아졌다. 조회 수·시점은 같고 대상만 달라졌다. 다른 12개는 면적도 동률이며 새 unaffected 양의 certificate도 없다. 독립 반복이나 실제 지연 절약이 아니다.

사전 지정한 B8/q3의 단계별 읽기

모든 아홉 정책·모든 시점·세 패널을 case_traces.json에 보관했다. 유리한 경로만 따로 생성하지 않았다.

1. t=0~3에는 기존·새 minimax one이 모두 조회하지 않는다. 2. t=4에는 둘 다 6785b851d6... 상태를 조회한다. 기존 trace에서 active lesson·memorized_development 연산 상태다. 3. t=5의 기존 one은 d4d6c80bd5...(suspended lesson·memorized operation)를 고른다. 이후 conditioned 폭 1/8, 사전 최악 union 폭 1/4다. 실제 저장 패널은 union 폭 1/8로 사전 최악을 달성하지 않는다. 4. 새 union-one은 a8cd6336a4...(lesson 없음·absolute operation)를 고른다. conditioned·사전 최악 폭은 모두 5/24다. 사전 목적에서 1/4보다 작지만, 실제 패널 폭을 직접 최소화한 정책이라는 뜻은 아니다. 5. 같은 시점 unrestricted safe 폭은 기존 7/8, 새 5/6이다. 한 구간 차이 1/24를 B=8로 나누면 면적 1/192다. 제한 모형 폭을 safe 폭으로 바꿔 읽지 않는다. 6. t=6 세 번째 조회 직후 unrestricted safe

폭은 7/12이며, 이후 t=8 최종 safe 폭은 둘 다 1/12다. 전체 면적은 기존 223/192, 새 37/32다. 7. late는 t=6,7,8에 조회해 최종 폭 1/24와 t=8 양의 certificate를 얻지만 면적은 75/64로 더 크다. 기다림의 tradeoff는 그대로 남는다.

검산과 재현

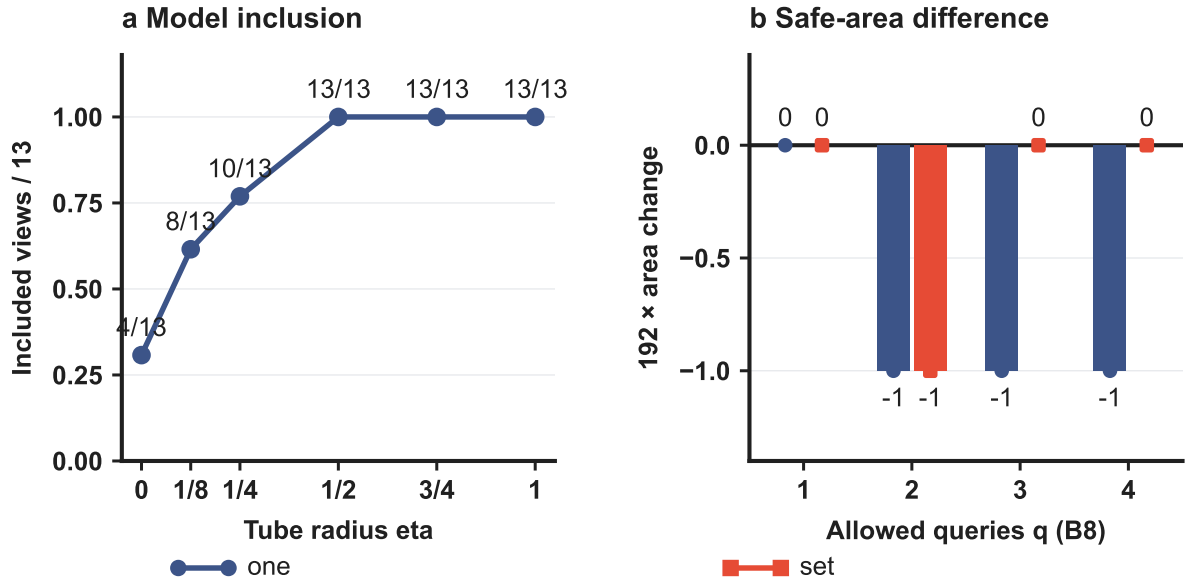
PLAN·analyze·verify 및 입력 173개(trace 168개·support 5개)는 새 출력에서 해시를 봉인한 뒤 분석했다. 원래 v17/v18과 기존 실행은 바꾸지 않았다. 아래 출력은 현재 존재하므로 그대로 재실행하면 덮어쓰기 대신 거절한다.

python projects/research_advancement_v19_20261019/second_chance/analyze.py --out outputs/research_advancement_v19_20261019/second_chance_analysis_01 python projects/research_advancement_v19_20261019/second_chance/verify.py --run outputs/research_advancement_v19_20261019/second_chance_analysis_01 --out outputs/research_advancement_v19_20261019/second_chance_analysis_01/verification_01 python projects/research_advancement_v19_20261019/second_chance/verify_envelopes.py --run outputs/research_advancement_v19_20261019/second_chance_analysis_01 --out outputs/research_advancement_v19_20261019/second_chance_analysis_01/envelope_verification_01

별도 원시 구간·vertex·가중 중앙값 검산은 540결정·1,620 safe bound·1,620 union view·22,968 미지 효용 completion을 확인했다. 첫 검산이 실제 정책의 방문·열거 집합을 다루므로 계획의 모든 visible 집합을 완결하도록 별도 code/input lock의 보충 검산을 추가했다. 동결된 첫 분석기·검산기는 수정하지 않았다.

보충 검산은 cap≤4 visible 집합 1,104개·known-utility vertex 11,159개·tube 달성 6,624개·η 전체 순위 90개를 확인했다. 과거 시점까지 실제 조회 가능 여부를 별도 flag로 보존했다. 최대 사전 premium 1/3 집합은 과거 조회 불가능한 계획 집합이다. 도달 가능 집합의 최대 1/5도 반사실적 효용 검산이며 실제 패널 발생률이 아니다.

검산은 같은 저자가 다른 식으로 작성했으며 외부 동료심사가 아니다. 다음 실험은 새로운 자연 교정 이력, 완전 상태·고정 입력 반복 감사, 결과 전 동결한 모형·정책·예산, 실제 호출·실패·시간 원장을 필요로 한다. 이번 실행은 이 입장 조건을 닫지 않았다.



Dependent views; inclusion is descriptive. Area: union minus restricted.

그림 1: 이번 후속 분석

이전 연구의 원리·실험·문헌 해설 · v18 보존본

SecondChance v17 상세 해설: minimax가 더 안전해 보이는데 왜 기다리는 규칙보다 못했는가

이 해설은 v17 원고와 같은 판본이다. 새로 실행한 것은 이미 노출된 저자 설계 로그에서 감사 선택 규칙 7개를 비교하고, 강건하다고 부르는 목적의 미래 가정이 실제 로그를 포함하는지 확인한 개발 분석이다. 실제 새 감사·worker 호출·자연 모델 이력·독립 전향 stream은 모두 0이며 전향 감사 효과는 null이다. 아래 과거 배경의 "이번"은 해당 과거 판본 시점을 뜻한다.

1. 점수를 다시 채점하는 것과 현재 상태를 감사하는 것

SecondChance의 원래 문제는 평가기가 고쳐진 뒤에도 현재 구현·지속 교훈·나중에 들어온 정상 변경이 실제로 회복되었는가다. 저장 답변의 점수 정정, 후보 새 실행, 통합 현재 상태의 검사는 서로 다른 사건이다. v17은 회복 worker를 새로 개선한 것이 아니라, 어떤 상태를 감사하면 정책 대비의 근거가 빨리 좁혀지는지 연구한다. 실제 회복 결과는 이전 그대로다.

v15의 감사 순서는 전체 미래 경로를 알고 계산했다. v16은 그 정보를 없애고 관측 prefix만 쓰는 규칙을 만들었다. v17은 한 걸음 더 나아가 "최악의 미래에도 좋다"라는 말에 어떤 미래를 허용했는지를 묻는다. 작은 가정 집합에서 minimax인 것과 실제 알 수 없는 미래에 안전한 것은 다르다.

2. 자료·기회·정보 권한을 고정한 비교

기존 저자 템플릿 12개와 경로 168개, 완전 상태 ID 9개를 그대로 보존했다. neutral_joint와 selective_joint 두 arm의 대비는 horizon마다 24개 경로 기록이고 전체 48개다. 선택기는 현재까지 도착한 commit,그 완료 prefix의 계수 p,현재 arm 점유 차이 d,현재시점 t,알려진 horizon B,이미 조회한 상태 ID만 받는다. 미래 commit·최종 계수·상태 프로그램·숨은 효용은 전달하지 않는다.

질의 기회는 t=1,…,B이고 t=0에서는 조회하지 않는다. 각 checkpoint마다 이미 관측된 새 상태 하나만 조회하며 전체 cap은 q=1~4다. 한 상태의 저장 관측은 세 고정 panel에 재사용한다. 한 번의 "quote"는 실제 모델이나 worker를 새로 실행하는 비용이 아니라 과거 관측값을 읽는 모의 단위다. 복구 비용 축과 이 quote 예산,실제 지연은 같은 숫자로 합치지 않는다.

총 7정책×2horizon×4cap=56설정,420결정,1,260개 안전 구간을 계산했다. 모두 같은 소수 이력을 재사용한 의존적인 파생 행이다. 표본이 1,260개로 늘어난 것이 아니다. 이전 v16의 24개 종료 비교도 그대로 재현했다.

3. 무제약 미래에서 minimax가 새 규칙이 되지 않는 이유

현재까지 조회하지 못한 prefix 계수들의 절댓값 합은 남은 감사 불확실성이다. 남은 길이는 r=(B-t)/B다. 미래 두 arm이 서로 다른 새 상태로 언제든 이동할 수 있으면,그 미래만으로 최대 2r의 추가 폭을 만들 수 있다. 이미 본 상태를 지금 하나 조회해도 아직 도착하지 않은 새 상태는 감사할 수 없다. 따라서 무제약 최악의 종료 조건부 폭의 상한은 미조회 prefix 절댓값 합 + 2r이다. 미래가 관측 직후 임의로 빠르게 바뀔 수 있는 경우 새 상태 두 개로 그 값에 임의로 가까워질 수 있다. 이는 supremum/closure다. 양의 최소 commit 지연이 있다고 가정한 실행에서 정확히 그 최대가 달성된다고 주장하지 않는다.

한 상태 s를 지금 조회해 줄일 수 있는 양은 정확히 |p_s|다. 그래서 현재 한 번의 minimax 선택은 기존 prefix_abs와 같다. "minimax"라는 이름을 붙였다고 더 강한 새 알고리즘이 된 것이 아니다. 이 결론은 한 단계 목적에 관한 것이며,미래의 질의 기회까지 최적으로 배분했다는 보장은 아니다. 뒤의 late-prefix 비교가 바로 이 차이를 보여 준다.

4. 다른 결정을 만들려면 어떤 가정을 추가했는가

별도 제한 모형은 최종 계수를 $c(\lambda)=p+\lambda\cdot r\cdot d, 0\le\lambda\le1$로 둔다. 지금의 arm 점유 차이가 남은 기간의 공통 λ비율 동안 지속되고,그 뒤 두 arm의 차이가 상쇄된다고 본다. 미래에 어떤 실제 commit이 나올지는 읽지 않는다. 이미 관측한 p와 d에서 만든 가설적 연속선이다. λ의 확률분포를 적합하거나 최종값을 알려 주지 않았다.

각 고정 λ에서,미조회 상태의 이진 효용으로 생기는 구간 폭은 해당 계수의 절댓값 합이다. 이를 λ전체에서 최악으로 만들면 볼록성 때문에 양끝 λ=0,1만 비교하면 된다. 고정된 미래 계수 시나리오 안의 폭을 최악으로 만든 목적이다.
여기서 중요한 함정이 있다. 이것은 λ도 아직 모를 때 모든 λ의 구간을 합친 전체 식별 폭이 아니다. 이미 조회한 상태의 계수와 기여도 λ에 따라 움직일 수 있기 때문이다. 현재 선택기는 이 조건부 최악 폭을 비교에만 사용한다. 실제 부호 인증과 안전 구간에는 끝까지 기존의 무제약 미래 suffix를 사용한다. 작은 제한 모형의 숫자를 안전 인증으로 바꾸지 않는다.

5. 비교한 일곱 규칙

규칙	현재 선택 방식	핵심 제한
FIFO	먼저 보인 상태부터 조회	미래에 중요한 상태라는 보장 없음
prefix_abs	과거 계수 절댓값 최대	무제약 한 단계 minimax와 같지만 전체 일정 최적은 아님
current_disagreement	지금 arm 점유 차이 최대	지금의 차이가 계속된다는 보장 없음
persistence_abs	p+r·d 절댓값 최대	지속을 가정한 한 점 예측
제한 minimax-one	양끝 조건부 폭을 가장 줄이는 한 상태	즉시 이득이 0이면 멈춤
제한 minimax-set	현재 보이는 상태의 남은 cap 부분집합을 정확 열거하고 첫 상태 조회	그 순간의 정적 최적이며 새 상태·미래 일정의 전역 최적 아님
late_prefix	마지막 q개 공개 기회까지 기다린 뒤 prefix_abs	미래 내용을 보지는 않지만 확실성을 늦게 얻는 대가가 있음

미래 내용을 미리 아는 arrival-constrained oracle도 따로 계산하지만 실제 실행 정책이 아니라 최종 폭의 진단 하한이다. 모든 일반 정책은 같은 정보 권한과 공개 기회만 쓴다. cap은 상한이므로 유효한 후보가 없을 때 덜 쓰는 것도 허용하고 실제 사용량을 남긴다.

6. 제한 모형이 실제 다음 경로를 포함했는가

종료 직전까지의 view는 모두 13개이고 그중 9개가 실제 최종 계수 벡터를 제한선 위에 포함하지 못했다. 각 horizon의 t=0도 포함한 분모다. 실제 질의 가능한 비종료 checkpoint만 보면 11개 중 7개다. B=5에서는 t0~t3,B=8에서는 t0~t4에서 벗어났다. 종료 시점은 r=0이라 원래 prefix가 완료 경로와 같아지므로 포함이 자명하다. 그 종료 일치까지 "예측 성공"으로 세면 가정의 약점을 감춘다.
이 검사는 전체 상태 좌표 합집합에서 한다. 기울기가 있는 모든 좌표가 동일한 λ를 요구하는지,기울기 0인 좌표는 이미 최종 값과 같은지 확인했다. 제한선과 실제 최종 벡터의 최소 L1 거리도 정확한 분수로 계산했다. 최종 벡터는 이 사후 검사에서만 읽으며 선택기로 보내지 않는다.
9/13을 자연 모델의 가정 실패 확률이라고 부르면 안 된다. 두 aggregate 경로에서 나온 의존적인 노출 view다. 또한 이 제한 가정 위반이 현재 1,260개 안전 구간의 참값 포함을 깨지는 않는다. 안전 구간은 별도의 더 넓은 무제약 suffix를 썼기 때문이다. 실제 운영에서 완전 상태·고정 입력의 결정론 자체가 틀린 경우는 또 다른 문제다.

7. 각각 이득 0인 두 조회가 합치면 도움이 되는 반례

네 상태 a,b,c,d에 대해 모형의 양끝 계수를 (1/4,-1/4,0,0)과 (0,0,1/4,-1/4)로 두자. 처음 최악 폭은 1/2다. a만 감사하면 두 번째 시나리오의 폭은 그대로 1/2다. c만 감사해도 첫 번째가 그대로라 최악 폭은 줄지 않는다. 그런데 a와 c를 함께 감사하면 두 시나리오 모두 1/4가 남는다.
처음 c의 이득은 0이지만 a를 고른 뒤의 c 이득은 1/4다. 한계 이득이 커지므로 diminishing returns가 성립하지 않는다. 즉시 이득 0에서 멈추는 greedy가 놓칠 수 있는 이유다. 공개 adaptive-submodularity 논문의 조건부 보장을 반박한 것이 아니라,우리 목적에 그 가정이 자동으로 들어 있지 않다는 별도 반례다. 이 네 상태 구성은 새 자연 이력이 아니다.

8. 기다리기가 실제로 minimax보다 좋은 끝값을 만든 경우

B=8,q=3에서 late-prefix는 t6,t7,t8에 조회했다. 최종 안전 폭은 1/24,도착 제약 oracle 대비 regret은 0이고 unaffected panel의 양의 대비를 t8에 세 quote로 인증했다. 제한 minimax 두 규칙은 t4,t5,t6에 조회해 최종 폭 1/12,regret 1/24였고 기한 안에 부호를 인증하지 못했다. 조회 수는 둘 다 세 개다.

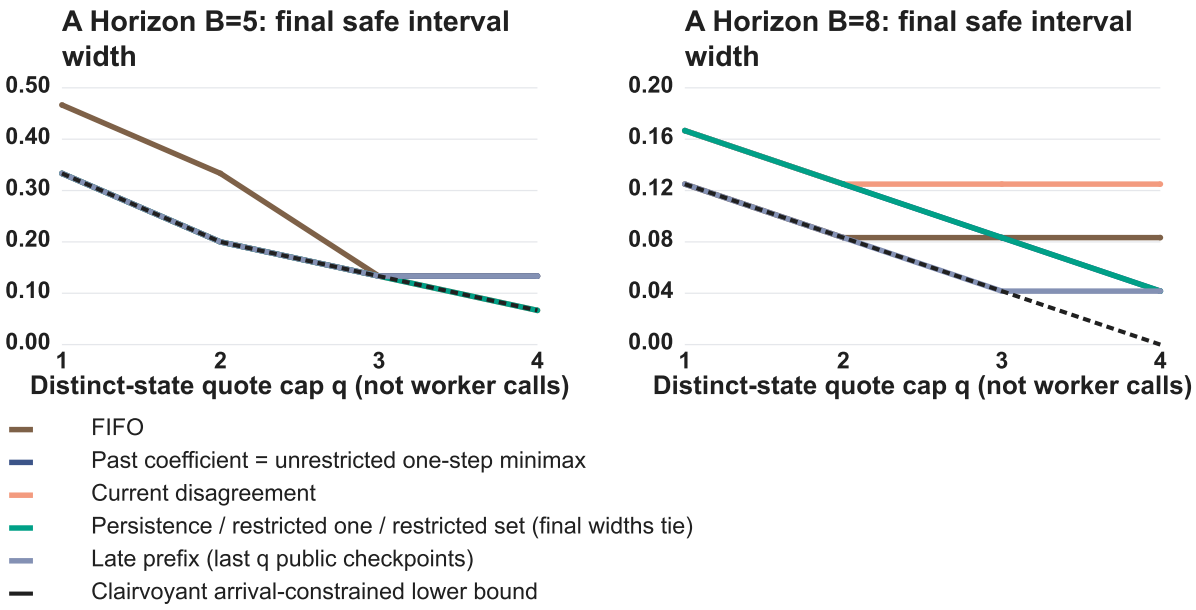
하지만 late의 checkpoint 폭 면적은 75/64,제한-set은 37/32다. 이 면적은 checkpoint0~B-1의 안전 폭을 합한 뒤 B로 나눈 이산 좌측 계단 요약이다. 중간에는 더 불확실하게 기다렸다는 뜻이다. 실제 지연초·연속 시간 기대 회복·통계적 신뢰수준이 아니다. 최종 폭 하나로만 순위를 매기면 이 대가를 빼먹는다.

B/q	규칙	조회 checkpoint	최종 폭	checkpoint 폭 면적	unaffected 인증
8/3	FIFO	1,2,4	1/12	37/32	미도달
8/3	제한-set	4,5,6	1/12	37/32	미도달
8/3	late-prefix	6,7,8	1/24	75/64	8
8/1	FIFO	1	1/8	75/64	미도달
8/1	late-prefix	8	1/8	19/16	미도달
5/4	현재 차이	2,3,4,5	1/15	6/5	5
5/4	제한-set	2,3,4,5	1/15	6/5	5

B=8,q=1에서는 FIFO가 t1,late가 t8에 조회하지만 최종 폭은 둘 다 1/8이다. 폭 면적은 FIFO75/64,late19/16이라 기다리는 규칙이 끝값 이득 없이 더 늦다. B=5,q=4에서는 현재차이·지속·제한minimax가 모두 폭1/15로 동률이다. 어떤 규칙도 보편 우위로 올리지 않는다.
제한-one·제한-set·단순 persistence는 8개 종료 비교에서 모두 같은 폭이었다. 다만 B=8,q=3에서 제한-set의 폭 면적은 제한-one보다1/192 작다. 추가 계획의 좁은 시점 차이는 있지만 새 종료 성능 향상으로 부르지 않는다. primary 선택에서 비어 있지 않은 부분집합483개를 열거했으며,별도 불변성 반복·검산·oracle 계산은 그 수에서 제외한다. 실제 CPU·모델 지연 절감을 주장하지 않는다.

SecondChance v17: conditional minimax is a comparator, not protection against every future

Same exposed authored logs / 12 templates / 56 dependent configurations. Actual new audit executions = 0.



B Does the actual final coefficient vector belong to the restricted continuation segment?

B=5	t0: out	t1: out	t2: out	t3: out	t4: in	t5: in		
B=8	t0: out	t1: out	t2: out	t3: out	t4: out	t5: in	t6: in	t7: in	t8: in

9 of 13 nonterminal views are outside (includes t0); these are dependent diagnostics, not a failure probability.

C At B=8 / q=3, later information narrows the final result but increases checkpoint uncertainty

Policy	Query checkpoints	Final width	Width area	Positive certificate
FIFO	1,2,4	1/12	37/32	not reached
Restricted-set	4,5,6	1/12	37/32	not reached
Late prefix	6,7,8	1/24	75/64	t8 / 3 quotes

그림 2: 가정 제한·안전 폭·기다림 비용을 나눈 감사 규칙 비교

논문 그림8이다. 위는 q별 종료 안전 폭,가운데는 제한선이 실제 최종 벡터를 포함하는지,아래는 B8/q3에서 동일 세 quote의 시점과 대가다. 서로 겹치는 persistence/제한minimax 세 곡선은 동률임을 명시해 묶었다. 초록·빨강 view 칸은 표본 독립성이나 확률을 뜻하지 않는다.

9. 중요한 비교 문헌을 어떻게 읽었는가

Adaptive Submodularity 원문 HTML v5 (https://arxiv.org/html/1003.3967v5)의 부분 관측 정책과 diminishing-returns 조건을 다시 확인했다. 우리는 prior·Bayesian posterior나 그 논문의 센서 실험을 재현하지 않았다. 이번 반례는 우리 robust 목적에 필요한 성질을 직접 검사한 것이다. 별도 가정 없이 greedy 근사 보장을 가져오지 않는다.

공식 COCO 평가 해설 (https://numbbo.github.io/coco-doc/perf-assessment/)에서 고정 예산 끝값과 목표 도달 시점의 구별을 다시 확인했다. 여기서는 미도달을 null로 보존하고 query 지출을 같이 남긴다. COCO benchmark 함수·optimizer 실행·외부 성능 수치를 우리 표에 합치지 않았다.

10. 재현과 다음 실제 실험

새 실행은 runs/research_advancement_v17_20261019_01/second_chance/analysis_01에 있다. PLAN.json,analyze/verify 코드,168 trace와2개 support를 결과 계산 전에 복사·해시 고정했다. v16 helper는 바이트 고정한 runtime/v16_prefix.py에서 view·기존규칙·구간·oracle 함수만 재사용하고 과거 run 함수를 호출하지 않는다.

작성자가 별도로 쓴 verifier는 새 scorer를 import하지 않고 단위 구간 적분,bitmask 부분집합,볼록함수의 모든 꺾이는 점으로 대조했다. 170입력해시,15prefix view,420선택,1,260구간,483부분집합,1,235breakpoint,8oracle 조건과 네상태 반례가 통과했다. 비저자 과학 검토와 PDF 조판·HTML 등록은 root가 별도로 수행한다.

실제 다음 실험은 새 commit이 오기 전에 선택기·기회·가정 모형을 고정하고,모든 visible view·실패·캐시hit·실제 호출·지연을 기록해야 한다. 완전 상태와 같은 입력에서의 반복 결과를 확인하고 drift가 있으면 재사용을 중단한다. 최종 폭만이 아니라 폭 면적·목표 도달 시점·미도달·실패 quote를 함께 비교한다. Li의 데이터·측정 관점과 Sutton의 예산·선택 관점 두 개가 현재 필요한 이유다. 실제 교수 개인의 자문·심사가 아니다.

아래는 v16까지의 원리·실행·문헌 검토를 보존한 과거 배경이다. 새 v17 결과와 합쳐 독립 실험 수를 늘리지 않는다.

SecondChance v16 상세 해설: 미래를 모르는 감사 순서와 증명의 한계

이번 판은 미래 commit을 보지 않는 세 감사 규칙을 실제 재생하고, 시점·조회 한도에 따라 순위가 뒤집히는 결과를 추가했다. 새 v16 내용은 17~24절이며 앞선 v15까지의 설명과 그림은 이력으로 보존한다. 독립 단위·실제 worker 실행·모델 API 호출은 추가하지 않았다.

v14의 빈틈에서 출발한다

v14는 9개 완전 상태의 512개 관측 부분집합을 모두 보아 어떤 조합이 결론을 확정하는지 계산했다. 그중 가장 적은 감사를 고르는 것은 전체 정답을 이미 본 사후 최적화다. v15는 그 숫자를 바로 스케줄러 성능으로 부르지 않고, 감사 점수를 아직 보지 않아도 정할 수 있는 순서를 실제로 구현했다.

새 연구의 입력은 이전 12개 저작 이력에 속한 168개 완전 경로다. 자연스러운 모델 실패 이력이나 새 교정 행동을 만들지 않았다. 새 worker 호출과 실제 감사 실행은 0개다. 기존 감사 결과를 필요할 때 하나씩 공개하는 오프라인 재생이다.

상태별 계수는 왜 생기는가

각 정책의 비용 정규화 면적은 완전 상태에 머문 시간 비율과 그 상태의 감사 점수를 곱한 합이다. neutral 평균 면적에서 selective 평균 면적을 빼면 상태마다 양수·음수·0의 계수가 남는다. 같은 완전 상태가 두 정책에 똑같이 나타나면 상쇄되므로 개별 정책 면적을 각각 다 알아야 차이를 아는 것은 아니다.

감사하지 않은 상태의 점수를 0 또는 1로만 알고 있을 때 그 상태가 만들 수 있는 대비 구간 폭은 계수의 절댓값이다. 아직 모르는 모든 상태의 절댓값을 더하면 전체 구간 폭이다. 상태 하나를 감사하면 해당 절댓값만큼 폭이 줄어든다. 점수 자체를 아직 보지 않아도 줄어드는 폭은 알 수 있다.

따라서 동일한 단가로 q개 상태를 감사한다면 절댓값이 큰 순서로 고르는 것이 폭을 가장 작게 만든다. 더 작은 계수를 고르고 큰 계수를 남긴 조합이 있다면 둘을 바꾸어 손해를 줄일 수 있다는 교환 논증이다. 이 성질은 단가가 다르거나 미래 경로가 아직 모를 때의 일반 최적 정책이라는 뜻이 아니다.

구현한 세 순서와 정보 권한

첫째 target_abs_coefficient는 각 budget에 필요한 대비 계수 절댓값 순서다. 둘째 cross_budget_l1_coefficient는 budget5와8의 절댓값을 합쳐 공통 순서를 만든다. 셋째는 state_hash

순서다. 모든 동률은 SHA-256 오름차순이다. 순서를 정할 때 initial/later/unaffected 감사 값은 읽지 않는다. 관측 값은 정지 여부를 판단할 때만 공개한다.
상태 하나를 새로 읽으면 고정된 세 패널 점수를 함께 공개한다고 정했다. 따라서 단위는 고유 완전 상태 한 개당 1quote다. 세 패널을 독립 비용 3으로 부풀리지 않고, 같은 상태가 여러 경로에서 재등장해도 같은 감사를 다시 비용에 넣지 않는다. 이 단가는 실제 모델 추론 시간·현금·산업 측정 비용을 재었다는 뜻이 아니다.

실제 결과: 구간 폭과 부호는 목표가 다르다

target 순서는 budget5 대비를 5회, budget8 대비를 4회에 정확히 식별했다. hash 순서는 각각 7회와8회가 필요했다. 두 budget을 동시에 배려하는 공통 순서는 5회와6회였다. 공통 목표를 위한 선택이 budget8 하나만 보면 더 비싸질 수 있다는 점도 남겼다.
initial/later의 참 대비는0이므로 엄격 양수·음수로 정지할 수 없다. 점 구간이 되어 동률을 확정해야 한다. unaffected에서는 target 순서가 budget5에서4회, budget8에서3회에 neutral의 엄격한 양수 우위를 증명한다. hash는5회와7회, 공통 순서는4회와4회다.
그런데 전체 결과를 알고 유리한 감사 집합을 사후 탐색하면 unaffected의 양수 증명은 두 budget 모두3회면 충분하다. budget5의 target 순서4회는 이 사후 최소3회보다1회 더 든다. 구간 폭 최소화는 가능한 값 전체의 불확실성을 줄이는 목표이고, 특정 실현 값의 부호를 빨리 증명하는 목표와 같지 않기 때문이다. 사후3회를 utility-blind 정책의 실측 비용으로 보고하지 않는다.

prospective라는 표현의 정확한 범위

이번 순서는 이 재생에서 감사 값을 공개하기 전에 고정된다. 그러나 모든 경로와 commit 시점은 이미 끝난 로그에서 가져왔다. 연구자는 이전 결과도 보았다. 따라서 '알려진 경로의 사후 감사 업무에서 점수를 보지 않고 정할 수 있는 순서'라고 해야 한다. 아직 발생하지 않은 미래 commit과 머무는 시간을 아는 실시간 정책이라고 부르면 정보 누출이다.
실시간 감사에서는 새 commit이 생길 때 계수가 바뀌고, 다음 commit까지 얼마나 머물지도 아직 모른다. 이 환경으로 옮기려면 현재까지의 로그만으로 결정하는 규칙, 미래 계수 불확실성, 실패·재시도·cache·실제 호출과 지연의 비용을 별도로 설계해야 한다. 이번에는 구현하지 않았다.

여전히 가장 중요한 실패 조건

완전 commit 로그, 완전한 상태 ID, 고정 입력, 순수 결정적 감사라는 전제가 깨지면 같은 상태 점수를 공유할 수 없다. 숨은 mutation 또는 같은 state/input의 다른 출력이 확인되면 좁은 구간을 정확한 결론으로 계속 제시해서는 안 된다. v14의 숨은 commit 반례는 그대로 유효하다.
필요한 관점도 두 가지로 한정한다. 데이터·측정 관점은 공유 상태 가정과 단위를 점검한다. 예산·탐색 관점은 폭 최소 목표와 부호 정지, 사후 최소와 실행 가능 순서를 구분한다. 공개된 연구 방법론을 연결하는 것과 당사자가 실제 조언했다는 것은 별개다.

검산

새 정량 그림은 논문 그림 6이다. 감사 수가 늘어날 때 세 순서와 두 budget의 직접 대비 구간 폭을 비교한다. 각 budget의 절댓값 순서와 모든 budget을 함께 배려한 순서는 같은 목표가 아니다.

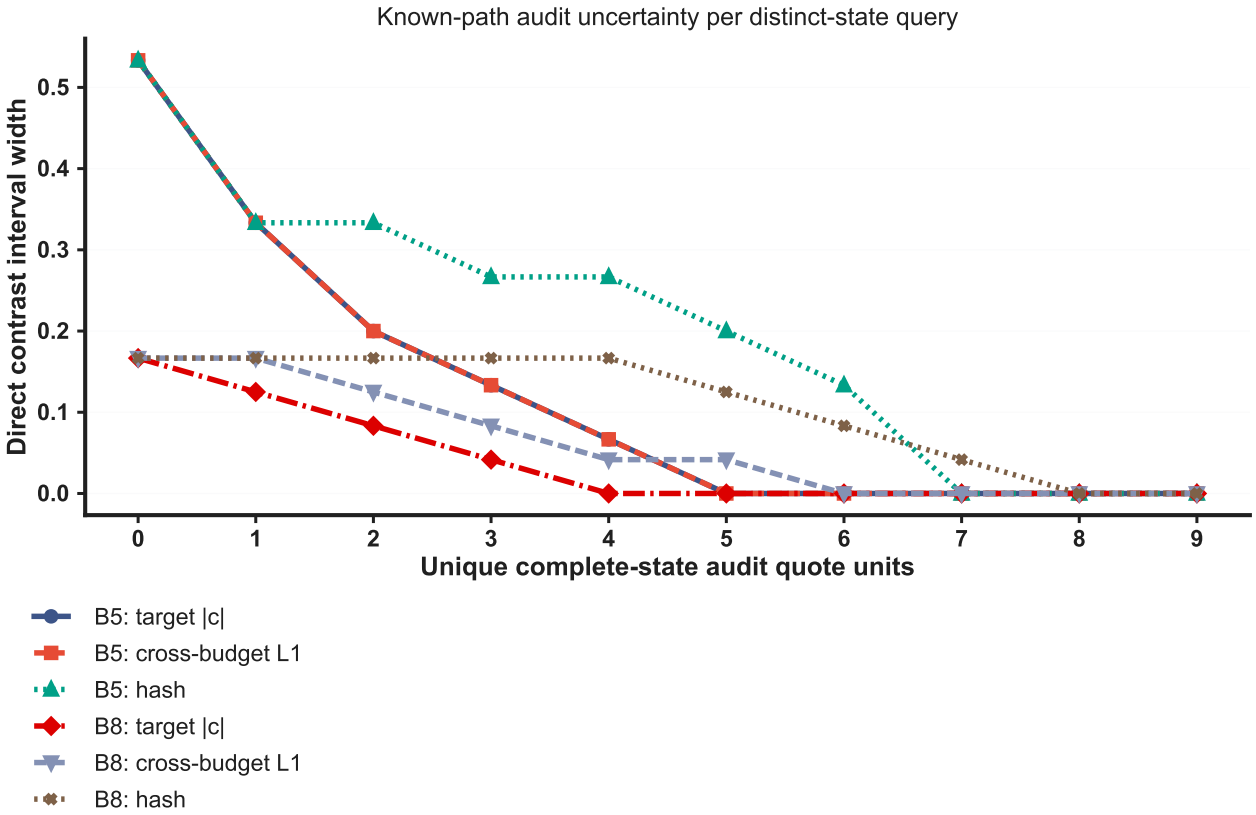


그림 3: 알려진 경로에서 감사 순서별 고유 상태 query 수와 직접 대비 구간 폭

모든 q=0..9와 두 budget에서20번의 전부분집합 폭 최소 검사를 통과했다. 180개 구간 모두 참값 포함과 단조 축소를 검사했다. 별도 verify.py는 원본168경로에서 계수를 다시 계산한 뒤 가능한 binary 완성18,414개를 열거하여180개 경계를 검산했다. 같은 저작 fixture의 대안 구현 검산이며 과학적 독립 재현은 아니다.

아래는 v14까지의 원리·문헌·이전 결과를 보존한 상세 배경이다.

SecondChance v14: 감사가 빠져도 정책 비교를 어디까지 믿을 수 있는가

이 해설은 v13의 세 endpoint·강한 neutral 기준·commit 면적을 유지하고, v14에서 실제로 실행한 통제된 관측 누락 분석을 설명한다. 원자료는 이미 공개된 저자 작성 결정론 환경의 12개 이력과 그에 중첩된 168개 경로다. 이번에 새 LLM 이력·새 정책·새 worker 응답을 만들지 않았다. 연구 질문을 '정책이 좋아졌는가'에서 '관측이 덜 남았을 때 어떤 비교까지 여전히 정당한가'로 확장했다.

1. 왜 이 질문이 필요한가

최종 checkpoint가 성공해도 그 전까지 얼마나 오래 실패했는지 알 수 없다. 반대로 성공 상태가 비용 구간 대부분을 차지했어도 마지막 실패 한 번만 보이면 전체 절차를 나쁘게 판단할 수 있다. v12는 완전한 상태 변경 기록과 새 감사로 이 간격 문제를 드러냈다. 그러나 실제 연구에서는 중간 감사가 실패하거나 기록되지 않을 수 있다. 그때 성공 여부를 임의로 0으로 채우거나 마지막 점수를 구간 전체에 복사하면 관측하지 않은 사실을 만들어 낸다.

이번에는 어떤 감사 결과를 알고 있는지 명시한 뒤 가능한 면적의 최솟값과 최댓값을 계산한다. 더 중요한 목표는 두 정책의 차이다. 두 정책이 같은 상태에서 같은 시간을 보냈다면 그 상태의 점수를 몰라도 비교에서는 상쇄될 수 있다. 개별 결과가 불확실하다는 이유만으로 모든 비교를 포기하거나, 반대로 독립적인 구간처럼 빼서 쓸데없이 넓은 범위를 만드는 문제를 검사했다.

2. 기존에 확인한 것과 이번에 추가한 것을 나누기

v11은 12개 작성 이력에 7정책과 예산 5·8을 적용해 168개 경로를 만들었다. v12는 그 고정 경로의 상태를 새 로컬 subprocess 292회로 감사해 initial/later/unaffected 876개 panel 출력을 관측했다. 초기 회복 면적은 16개 경로에서 성긴 관측보다 0.2 컸고, 비관련 능력의 면적은 32개 경로에서 작았다. 이 숫자는 유지한다.

v14는 그 결과에 감사를 남기거나 가리는 규칙을 적용했다. 고유 완전 상태는 9개이므로 관측 집합은 2의9승=512개다. 각 집합을 예산 2개와 panel 3개에서 비교해 3,072개 contrast bound를 계산했다. 이는 새 원응답 3,072개나 독립 실험 3,072개가 아니다. 같은 상태와 이력을 반복 사용한 유한 진단이다. 실행에는 약 12.89초가 걸렸지만 다른 감사 알고리즘과 시간을 통제해 비교하지 않았으므로 속도 우월성도 주장하지 않는다.

3. 세 endpoint를 다시 정확히 정의하기

E1은 같은 저장 답을 교정된 평가기로 다시 채점하는 것이다. E2는 선택한 후보를 실제로 다시 실행하는 것이다. E3는 후보·교훈·보존해야 할 후속 변경까지 합친 현재 checkpoint를 새 입력에 실행하는 것이다. 저장 답의 점수 상승은 현재 파일이나 교훈을 자동으로 바꾸지 않는다.

작성 환경의 원래 요구는 두 분수의 정확한 합이다. 잘못된 평가기는 합의 절댓값을 정답으로 취급했고, 교정 평가는 부호를 포함한 원래 합을 사용한다. 예산8에서 rescore-only는 저장 점수 12/12를 올렸지만 현재 초기 회복은 0/12였다. selection-only는 새 개발 입력 후보 성공 12/12였지만 현재 초기 회복 4/12, 다른 입력 panel 성공 2/12였다. 요구가 바뀐 사건을 '원래 평가기 교정'에 자동 편입해서는 안 된다.

4. 면적의 단위와 상태 점유 가중치

경로의 비용 축을 0부터 B까지 두고, 구간 j에서 상태가 s이면 그 상태의 효용 u(s)에 구간 길이를 곱한다. 전체 B로 나누면 정규화된 면적이다. 비용4에서 처음 성공하고 B=5까지 유지되면 성공 면적은 1/5다. 비용5인 마지막 순간에만 성공 상태가 생기면 최종 성공은 바뀌지만 그 뒤 지속 시간이 없으므로 면적 증가는 0이다.

같은 상태가 여러 구간에서 다시 등장하면 각 구간 길이를 합친다. 같은 상태로 남는 metadata commit은 새 점수 변수를 만들지 않는다. 열두 이력에 같은 가중치를 주면 정책 p의 상태별 점유 가중치 w(p,s)를 얻는다. 각 정책의 가중치 합은 1이고 A(p)=Σ_s w(p,s)u(s)다. 여기서 비용은 작성된 work unit이며 토큰·돈·실제 CPU 시간이 아니다. 감사 작업량을 과거 정책 비용 축에 소급해서 더하지도 않는다.

5. 무엇을 알고 무엇을 모른다고 가정했는가

주된 분석은 모든 commit 시각과 상태 identity를 알고 있지만 일부 상태의 감사 효용만 모르는 정보 모형이다. 감사를 남긴 상태는 세 고정 입력의 점수를 함께 공개한다. 같은 완전 상태·같은 worker·같은 입력이면 같은 점수라는 순수 결정론 가정을 사용해 다른 경로에서도 그 관측을 재사용한다.

이것은 연구자가 원프로그램을 읽어도 점수를 절대 알 수 없다는 주장이 아니다. 전체 작성 코드와 전체 결과는 연구자에게 노출돼 있다. 이번 계산기는 저장된 감사만 허용하는 정보 모형을 따르고, 가린 응답을 산술 코드에서 새로 추론하지 않는다. 따라서 자료 누락에 대한 민감도 실험이지 실제 현장의 비밀 평가나 독립 final이 아니다. 코드 의미·허용 프로그램 집합·panel 간 제약까지 이용하면 구간은 더 좁아질 수 있다.

6. sharp bound를 직접 유도하기

두 정책을 neutral n과 selective q로 두고 c(s)=w(n,s)−w(q,s)라고 쓰면 차이는 D=Σ_s c(s)u(s)다. 관측한 상태 O의 기여 K=Σ_{s∈O}c(s)u(s)는 고정이다. 미관측 상태 M의 각 u(s)는 0 또는1이다.

c(s)가 양수이면 최소를 만들 때 0, 최대를 만들 때 1로 둔다. 음수이면 반대로 둔다. 그래서 하한은 K+Σ_{s∈M}min(c(s),0), 상한은 K+Σ_{s∈M}max(c(s),0)다. 각 상태에는 값을 한 번만 배정하므로 정책 사이의 동일 상태 제약을 지킨 채 양끝을 실제로 달성하는 completion을 만들 수 있다. 이것이 이 정보 모형에서 sharp하다는 뜻이다.

binary 변수이므로 가능한 차이의 집합은 띄엄띄엄할 수 있다. 하한과 상한 사이 모든 실수가 실현된다는 뜻이 아니다. 이번 보고는 가능한 집합의 sharp interval hull이다. 또한 표본 추출 확률을 가진 신뢰구간이 아니다. 누락이 무작위라는 가정도 하지 않는다. 같은 저자가 만든 12개 템플릿에 population 95% CI를 붙이는 연구가 아니다.

7. 개별 면적 두 개를 빼면 왜 느슨해지는가

가장 간단한 예는 두 정책이 같은 미관측 상태에서 전체 시간을 보내는 경우다. 각 면적은 0부터1까지 가능하다. 두 면적을 별개로 빼면 차이가 −1부터1까지라고 나오지만, 같은 상태에 같은 u를 사용하면 실제 차이는 언제나0이다. 서로 다르게 할 수 없는 미지수를 따로 극단화한 것이 문제다.

정책 차이의 계수 c(s)=0인 상태는 미관측이어도 차이에 영향을 주지 않는다. 이 단순 성질이 실제 168개 경로에서도 얼마나 남는지 계산했다. 다만 상태 해시가 같은 것만으로 순수성을 인증할 수는 없다. 파일·도구·retriever·외부 서비스·숨은 memory가 상태에 빠졌거나 응답이 확률적이라면 같은 hash의 방문 두 번에 다른 효용이 나올 수 있다. 방문별 값을 neutral0/selective1로 두면 차이−1, 반대로 두면+1이므로 cancellation이 무너진다. 이 예는 실제 LLM에서 관측한 사건이 아니라 가정의 필요성을 보여주는 수학적 반례다.

8. 새 그림 4: 관측이 일부 빠져도 neutral 우위가 식별되는 경우

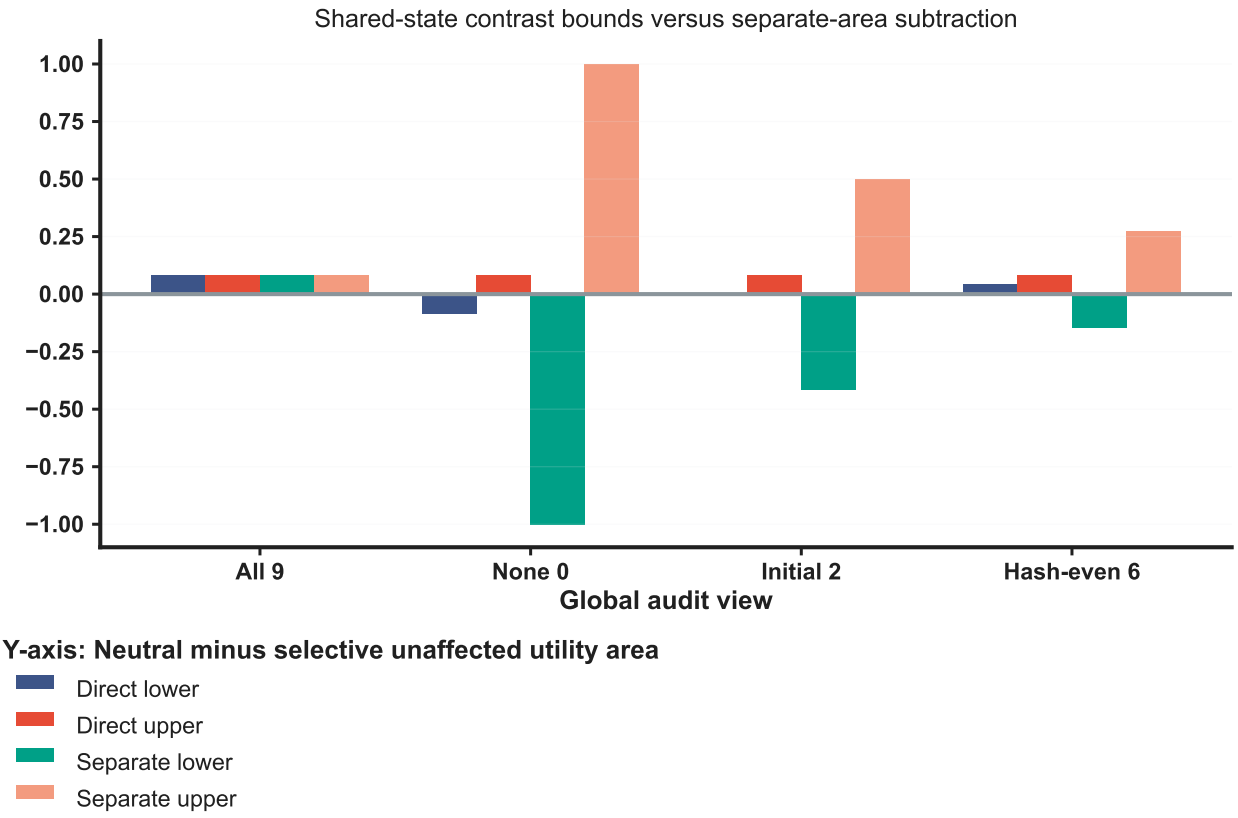


그림 4: 예산8 비관련 효용의 직접 bound와 별도 면적 구간 빼기

예산8의 비관련 과제를 보자. 전체 9개 상태를 관측하면 neutral−selective 면적은 정확히1/12다. 아무것도 관측하지 않으면 직접 bound도 −1/12부터1/12라 양쪽 부호가 가능하다. 그래도 별도 면적을 빼서 얻는 −1부터1보다는 좁다.

초기 상태 2개만 남기면 직접 bound는 0부터1/12가 되어 neutral 비열등성까지는 말할 수 있지만 엄격 우위는 아니다. 사전에 정한 hash-even 규칙으로 6개 상태를 남기면 1/24부터1/12가 된다. 하한이 양수이므로 이 모형에서는 neutral의 엄격 우위가 식별된다. 반면 개별 면적을 따로 bound해 빼면 −7/48부터13/48이라 우열을 결정하지 못한다. 그림의 막대는 구간의 양끝 값이며 각 막대가 독립 성능 실험인 것은 아니다.

예산5의 같은6상태 관측에서는 초기와 다른 입력의 정책 차이가 정확0인데 별도 면적 빼기는 −1/15부터1/15다. 이는 불확실성을 정직하게 처리하면서도 불필요하게 넓히지 않는 예다. selective 정책의 개선을 만든 결과가 아니라, 기존의 동률과 neutral의 부수 손실 우위를 더 정확히 해석한 결과다.

9. 새 그림 5: 512개 관측 집합을 어떻게 읽어야 하는가

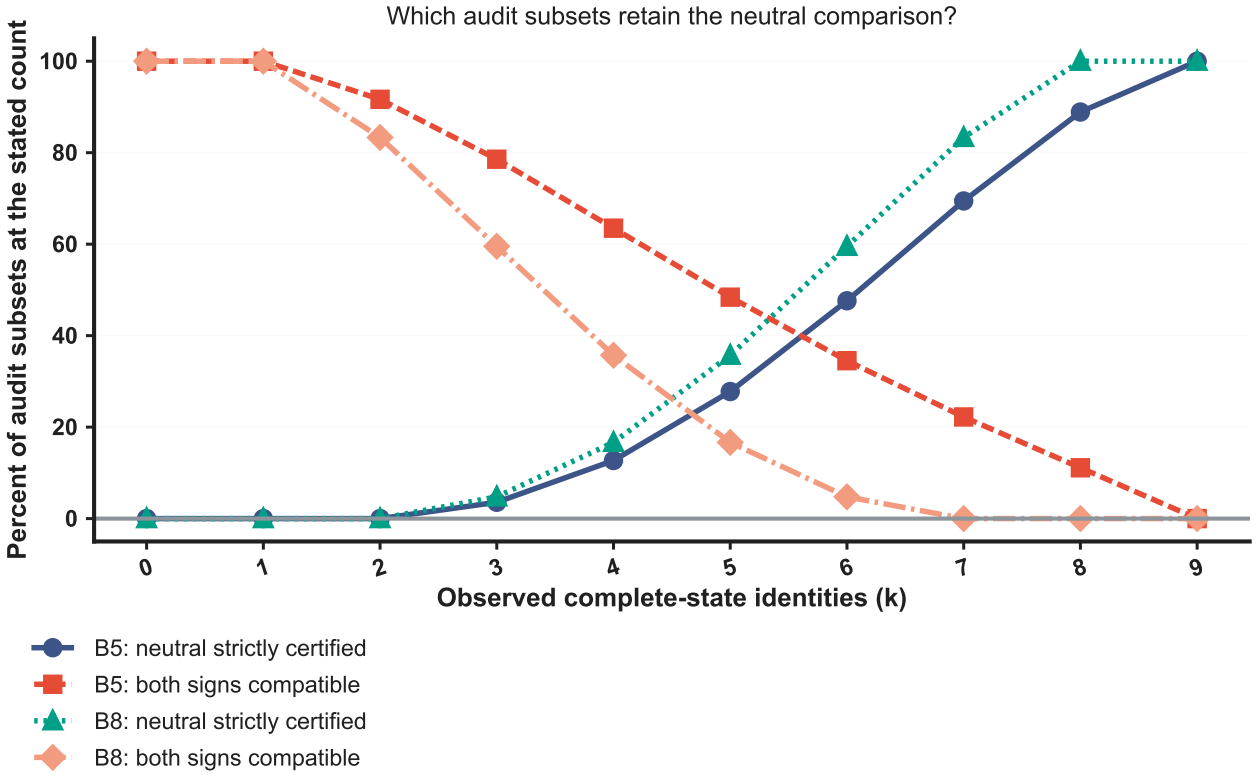


그림 5: 관측 상태 수별 정책 비교의 식별 가능성

상태 k개를 관측하는 집합의 수는 choose(9,k)다. k=0부터9까지 분모는 1·9·36·84·126·126·84·36·9·1이다. 같은 k 안에서 엄격 neutral 우위가 증명되는 집합, 양부호가 모두 가능한 집합 등을 센다. 이 곡선의 비율은 조합적 비율이다. 현장에서 누락이 그 집합들에 균등하게 발생한다는 확률을 추정한 것이 아니다.

전체512집합을 합치면 비관련 과제의 엄격 neutral 우위는 예산5에서128집합, 예산8에서160집합에서 확인된다. 양부호 모호성은 각각288집합과160집합에 남는다. 초기·다른 입력의 정확한 동률은 예산5에서16집합, 예산8에서32집합이며 나머지 일부는 약한 한쪽 부호만 허용한다. 약한 비열등성과 엄격 우위를 구분하지 않으면 이 진단도 과장된다.

각 panel에서 직접 interval이 별도 면적 빼기보다 엄격히 좁은 집합은 예산5의480/512, 예산8의510/512다. 개별 면적 중 하나 이상이 아직 불확실해도 차이만 점식별되는 집합은 각각12개와30개다. 이 역시 많은 독립 표본에서 발견한 빈도가 아니라 같은9상태 모형의 전수 결과다.

10. 최소 감사 증명집합과 실제 알고리즘은 다르다

정확한 정책 차이를 얻는 최소 관측 상태 수는 예산5에서5개, 예산8에서4개였다. 두 정책 면적을 각각 정확히 아는 데에는7개와8개가 필요하다. 비관련 효용의 neutral 엄격 우위만 증명하려면 두 예산 모두3개로 가능한 집합이 있다. 초기·다른 입력은 실제 차이가0이므로 어떤 정직한 관측 집합도 엄격 우위를 증명할 수 없다.

이 최소값은 전체 감사 결과를 알고 나서 모든 집합을 확인한 사후 certificate다. 어떤 상태를 새로 감사해야 할지 미리 모르는 실행기에게 같은 절감이 가능한지는 별도 문제다. 이번에 292회 중 일부 호출을 실제로 생략해 성공한 것도 아니며 새 audit scheduler를 평가한 것도 아니다. 다음 단계에서는 관측값을 보기 전 선택 규칙·비용·중단 기준을 고정한 scheduler를 별도 예산으로 비교해야 한다.

11. terminal-only·coarse-only가 모두 식별되는 예상 밖의 이유

이번 자료에서는 여러 경로의 terminal 상태를 전역으로 모으면 고유9상태가 전부 등장했다. 이전 coarse grid에서 나타난 상태도 전역으로 모으면9개 전부다. 주된 분석이 완전한 commit 시각을 계속 알고 있으므로, 이 전역 관측은 all-observed와 같은 면적을 재구성한다.

이것은 성긴 격자의 left-step 면적이 옳다는 뜻이 아니다. 다른 경로에서 이미 관측한 동일 상태의 효용을 가져오고, 여전히 알고 있는 정확한 commit 시각을 사용했기 때문에 가능한 일이다. 시각까지 빠져 있으면 이 복원이 성립하지 않는다. 같은 상태를 한 경로에서 가렸다고 하면서 다른 경로의 공개된 같은 점수를 무시하면 인위적 모호성을 만들게 된다. v14는 그래서 누락을 전역 상태 기준으로 정의했다.

12. commit 기록도 불완전할 때의 불가능 범위

유한한 checkpoint만 알고 그 사이 임의의 숨은 연속시간 상태 변경을 허용하면 상황이 달라진다. 각 관측 시점의 아주 짧은 근방에서만 그 관측값을 맞추고, 중간 대부분을 효용0 또는1로 채운 두 경로를 만들 수 있다. 간격의1/1000씩을 양끝 근방에 배정하면 모든 관측이 같은데도 낮은 면적은0.002 이하, 높은 면적은0.998 이상이다. 기존168경로×3panel=504개의 관측 모양 모두에서 이 구성의 조건을 검사했다.

근방을 더 줄이면 sharp한 폐구간은0부터1까지다. 오른쪽 연속 상태가 양의 시간 지속돼야 한다면 정확한0이나1을 달성하지 못할 수 있으므로 infimum/supremum이라고 표현한다. 부분 기록이면 무조건0~1이라고 말하는 정리는 아니다. 이 반례는 integer-only commit 격자·최소 체류시간·숨은 변경 횟수 제한·허용 전이 그래프를 제거한다. 그런 조건이 남아 있다면 더 좁은 bounds를 따로 계산해야 한다. 새 후보나 worker를 실행한 경로도 아니다.

13. v13의 불리한 결과를 v14가 지우지 않는다

예산5에서 neutral joint와 selective joint의 초기·later 성공은 모두4/12다. 그러나 neutral의 평균 작업량은3, selective는5이고 최종 unaffected incremental loss는0 대2/3이다. 비용 축에 적분한 unaffected 면적은1 대13/15로 차이2/15다. 최종 손실2/3과 면적 차이2/15는 다른 지표이므로 서로 바꿔 쓰지 않는다. 예산8은 성공4/12로 같고 실제 지출도 서로 다르다.

같은 artifact의 교훈 유지/중지12쌍 중2쌍이 later-input에서 나아졌던 결과도 남는다. 그러나 이 harmful lesson과 routing은 저자가 작성했다. 이후 새 진화가 없는 같은 checkpoint의 다른 입력 panel이므로 장기 재발률이 아니다. 초기 성공이0이면 그 성공에 조건부인 유지 실패율도 null로 남겨야 한다. 관측 누락 분석이 이 endpoint 정의를 바꾸지는 않는다.

14. 원문 문헌과 이번 수학의 관계

v13의19개 문헌과 원문 검토 범위는 해당 SOURCES 해시로 계승했다. SkillOpt·Reflexion·Self-Refine·CRITIC 등과 native 성능 대결을 새로 한 것은 아니다. 새로 확인한 원문은 Imbens와 Manski의 2004년 Econometrica 논문의 공식 출판사 metadata·초록이다. 이 초록은 identification region과 confidence coverage를 구분하는 데만 사용한다. 본문 증명·균등 수렴 조건·실제 CI 알고리즘을 검토하거나 구현했다고 쓰지 않는다.

이번 부호별 극단 배정은 유한 선형식의 초등적 도출이다. 문헌을 붙였다고 새로운 정리의 우선권이나 외부 인증이 생기지 않는다. SOURCES의 review-record SHA는 검토 메모의 정체성을 가리키고 출판사 원응답 byte hash는 미보관이라 null이다. 해시 일치와 참·과학적 독립성은 다르다.

15. 검증과 재현 순서

1. PLAN.json에서 정보 모형·누락 규칙·분모·실행 경계를 먼저 읽는다. 2. source_receipt.json에서 원본177개와 snapshot, 실제 소비 코드·계획의 해시를 확인한다. 3. state_catalogue.json의9상태·고정 panel 결과와 원 v12 trace를 대조한다. 이 파일은 완전 oracle이며 가린 계산기에 모두 공개한 입력이 아니다. 4. audit_subset_bounds.jsonl에서 원하는 mask·예산·panel의 fraction 양끝과 분류를 확인한다. contrast_views와 certificates는 읽기 쉬운 요약이다. 5. verify.py로 원시 구간을 독립 재집계하고, test_bounds.py로 terminal zero-width·동일 상태·부호·불량 시각·숨은 commit 반례를 검사한다.

계산식과 모든 binary completion의 양끝 일치3,072건, true contrast 포함3,072건, 직접 interval의 포함 관계3,072건, 양끝 실현6,144건, 관측 추가 시 단조 축소13,824건을 확인했다. 이 숫자는 유한 소프트웨어 관계의 검증이며 모델 효과나 과학적 타당성 검증 횟수가 아니다. 원래 v12의 실패 실행과 기존 결과는 수정하지 않았고 v14도 기존 출력 폴더를 재사용하지 않는다.

16. 다음 확증 실험과 중단 기준

자연 모델 연구에서는 먼저 실제 옛 평가기의 reject 이력·후보 archive·당시 생성된 교훈·보존할 후속 변경이 있는 사례를 모아야 한다. 고정 요구 아래의 교정인지 독립적으로 승인하고, 평가 권한·예산·누락 처리·중단 기준을 결과 전에 고정한다. 강한 neutral과 current-only regeneration을 같은 권한으로 비교한다. 유용한 reject가 없는 사례와 교훈이 없는 사례를 분모에서 지우지 않는다.

감사 연구를 확증하려면 전체 상태·환경 snapshot을 바탕으로 같은 상태를 반복 실행해 deterministic equality가 성립하는지 또는 기대값 모형이 필요한지부터 판별한다. 실제로 관측을 덜 획득하는 사전 정의 scheduler를 random 또는 fixed audit schedule과 동등 비용으로 비교할 수 있지만, 이것은 이번 사후 certificate와 다른 실험이다. 숨은 상태·상태 불일치·관측 누락이 발견되면 점식별 주장을 중단하고 그때의 정보·전이 제약에서 bounds를 다시 계산한다. 장기 재발은 복구 뒤 새 진화 prefix를 추가한 별도 연구로 둔다.

이번 진전은 더 좋은 점수를 만들었다는 주장이 아니라 어떤 비교를 계속 주장할 수 있고 언제 주장을 중단해야 하는지 계산 가능한 계약으로 만든 것이다.

17. v16: 미래 commit을 모르는 감사기가 실제로 할 수 있는 일

이번 v16에서 가장 크게 바뀐 점은 감사 순서를 결정하는 함수가 미래 경로를 받지 않는다는 것이다. v15는 효용값을 보지 않았지만, 모든 commit의 시각과 최종 경로를 알고 계수를 계산했다. 이를 실제 진행 중인 연구에 그대로 적용하면 미래에 어떤 상태가 오래 유지될지 이미 안다는 문제가 있다. v16은 그 조건을 제거한 실행 가능한 세 규칙을 같은 저장 로그에서 재생했다. 새 실제 worker를 실행하거나 새로운 실패 사례를 수집한 것은 아니다.

복구 비용 한도 B는 5 또는 8이다. 정수 시점 t=1부터 B까지, 그때까지 도착한 commit만 공개한다. 감사기는 매 시점 최대 한 상태를 조회하고 전체 조회 한도 q는 1·2·3·4다. 이 '한 시점에 한 번'은 미리 정한 기회 제약이다. 모델이 한 비용 단위 동안 한 번 추론한다는 실측이나, 감사 지연을 측정한 값이 아니다. 복구 경로는 고정되어 있으며 감사한다고 멈추지 않는다. 감사 조회 비용을 원래 복구 예산 B에서 빼지도 않는다. 복구 비용축과 감사 조회 수는 다른 자원이다.

정책이 받는 정보는 현재 시각, 현재까지의 상태 해시·commit 시각, 이미 완료된 구간의 상태별 점유 계수, 지금 활성 상태의 두 arm 간 빈도 차이, 앞서 조회한 상태 집합뿐이다. 최종 경로·최종 계수·아직 읽지 않은 효용·상태 프로그램의 의미는 전달하지 않는다. 상태를 선택한 뒤에만 그 상태의 기존 initial/later/unaffected 점수를 꺼내 bound를 계산한다. 재생 평가자는 전체 정답을 알아도 되지만, 정책 함수의 정보 권한은 그보다 작다.

18. 세 가지 인과적 규칙과 서로 다른 약점

- first_seen/FIFO: 먼저 도착한 미조회 상태부터 읽는다. 도착 시점이 같으면 해시 순서로 정한다. 현재 차이에 도움이 없어도 먼저 온 상태를 읽을 수 있다.
- prefix_abs: 완료된 과거 구간에서 두 arm의 점유 차이 절댓값이 가장 큰 상태를 읽는다. 유효한 차이가 없으면 건너뛴다.
- current_disagreement: 지금 두 arm에서 사용하는 상태의 빈도 차이가 가장 큰 상태를 읽는다. 지금 상태가 앞으로도 유지된다고 가정하는 bound가 아니라, 다음 조회의 우선순위일 뿐이다.

prefix_abs는 과거를 충실히 보지만 미래에 중요해질 상태를 놓칠 수 있다. current_disagreement는 지금 갈라진 상태를 빨리 보지만 과거 누적 효과를 놓칠 수 있다. FIFO는 단순해서 불필요한 상태에 조회를 쓸 수 있지만, 우연히 앞으로 중요해질 상태를 먼저 읽을 수도 있다. v16의 핵심은 특정 규칙을 무조건 옹호하는 것이 아니라 이 차이를 같은 권한·같은 도착 시점·같은 조회 기회에서 측정하는 것이다.

19. 아직 오지 않은 미래를 bound에서 지우면 안 된다

완료된 과거 구간의 계수 d_s(t)는 [0,t) 동안의 두 arm 점유 차이를 최종 예산 B로 나눈 값이다. t로 다시 정규화하지 않는다. 그래야 관측한 부분과 남은 부분을 합쳤을 때 원래 최종 면적 차이가 된다. 이미 조회한 상태 집합을 O_t라 두면, 알려진 부분의 합 K_t에 미조회 과거 계수의 양·음 부분을 더해서 과거 bound를 얻는다.

여기에 미래 반경 r_t=(B-t)/B를 반드시 더해야 한다. 각 arm의 앞으로 남은 효용은 0부터 r_t까지 가능하므로 차이는 -r_t부터 +r_t다. 따라서 아래끝은 K_t + 미조회 음의 계수 합 - r_t, 위끝은 K_t + 미조회 양의 계수 합 + r_t다. 아직 나타나지 않은 상태를 저장 파일에서 최종적으로 발견한 9개로 제한하지 않는다. 중간 시점의 구간은 안전한 외부 bound이며, 별도 전이 제약을 반영한 가장 좁은 구간이라고 주장하지 않는다. 마지막 t=B에는 미래 반경이 사라져 기존의 sharp audit-only 구간으로 돌아간다.

같은 상태·같은 고정 입력이면 효용이 같다는 가정은 계속 필요하다. 숨은 메모리, 환경 변화, 불완전한 state snapshot 때문에 같은 해시의 재실행 값이 바뀌면 이 재사용은 성립하지 않는다. 계산식의 정확성과 현장 상태 동일성은 다른 검증이다.

20. 먼저 알 수 없다는 것을 보이는 두 세계

B=4, 현재 t=1이라고 하자. 지금까지 두 arm은 효용 1/2인 동일 상태에 있었고 그 상태를 완전히 감사했다고 하자. 현재까지의 모든 로그가 같은 두 세계를 만들 수 있다. 첫 세계에서는 앞으로 비용 2에서 neutral이 효용 1, selective가 효용 0인 상태로 바뀐다. 둘째 세계에서는 그 반대가 된다. t=1에서는 두 세계를 구별할 수 없지만 최종 면적 차이는 각각 +1/2와 -1/2다.

따라서 과거 상태를 모두 감사했다는 이유만으로 최종 부호까지 확정할 수는 없다. 계산된 미래 포함 구간 [-3/4,3/4]는 두 경우를 모두 담는다. 이 예는 중간 구간이 가장 좁다는 증명이 아니라, 미래를 제거하면 안 된다는 반례다. 이 두 세계는 별도의 수학적 구성으로, 실험에 새로운 두 개의 독립 경로를 추가한 것이 아니다. 허용 전이·최소 체류 시간·확률 모형이 주어진다면 별도 연구로 더 좁힐 수 있다.

21. 새 그림 7: 과거 계수 greedy가 항상 좋은 것은 아니다

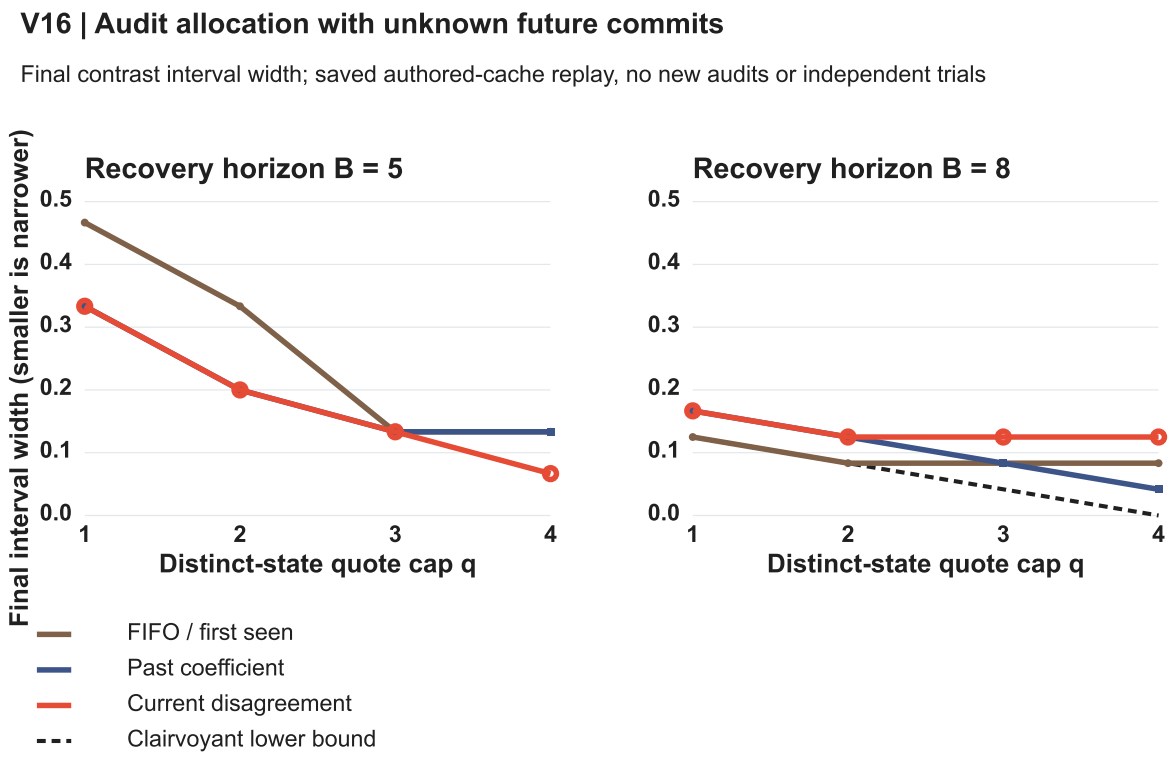


그림 6: 미래를 모르는 세 감사 규칙의 최종 불확실성과 미래를 아는 하한

세로축은 최종 neutral-minus-selective 차이 구간의 폭으로, 작을수록 남은 불확실성이 적다. 가로축은 조회 상한이다. 점선 oracle은 최종 계수를 미리 아는 계산상 하한이다. 상태가 나타나기 전에 조회하지 못하고, 매 시점 한 번이라는 기회 제약도 똑같이 지킨다. 그런데 무엇이 나중에 중요해질지는 미리 안다. 그러므로 실전에 투입할 수 있는 비교 정책으로 소개하면 안 된다.

복구 예산 / 조회 한도	FIFO 폭	과거 계수 폭 [실제 조회]	현재 차이 폭 [실제 조회]	미래를 아는 하한
5 / 1	7/15	1/3 [1]	1/3 [1]	1/3
5 / 2	1/3	1/5 [2]	1/5 [2]	1/5
5 / 3	2/15	2/15 [3]	2/15 [3]	2/15
5 / 4	2/15	2/15 [3]	1/15 [4]	1/15
8 / 1	1/8	1/6 [1]	1/6 [1]	1/8
8 / 2	1/12	1/8 [2]	1/8 [2]	1/12
8 / 3	1/12	1/12 [3]	1/8 [2]	1/24
8 / 4	1/12	1/24 [4]	1/8 [2]	0

B=5, q=4에서는 현재 차이 규칙이 폭 1/15로 하한을 달성하고, 과거 계수 규칙은 폭 2/15에 머문다. 과거 계수 규칙이 실제로 쓴 조회는 3번이다. 앞선 기회에 보이는 남은 계수가 상쇄되어 건너뛰었기 때문이다. 쓰지 않은 크레딧을 나중에 쓴 것으로 채우지 않았다.

반대로 B=8에서 한 번만 읽을 때는 FIFO의 폭 1/8이 두 greedy 규칙의 1/6보다 작다. 과거 계수 규칙이 단순 FIFO보다 불리한 반례가 실제 저장 로그 재생에서 나왔다. q=4가 되면 과거 계수 규칙은 1/24로 FIFO 1/12보다 낫지만 oracle의 0에는 도달하지 못한다. 즉 cap에 따라 순위가 바뀐다. 최종 계수가 고정되어 있을 때의 v15 교환 논증으로 미래 계수가 바뀌는 v16의 최적성을 주장할 수 없다.

22. 좁은 구간, 부호 증명, 실제 감사 시간은 다르다

24개 재생 설정 중 unaffected의 양의 차이를 증명한 것은 두 설정이다. B=5/q=4의 current_disagreement가 시점 5에, B=8/q=4의 prefix_abs가 시점 8에 각각 4번 조회 후 증명한다.

initial과 later의 실제 차이는 0이며, 이번 한도 안에서는 어느 규칙도 정확한 동률 구간까지 좁히지 못했다. 증명에 도달하지 못한 설정은 첫 시각과 그때의 조회 수를 null로 남겼다. 이를 0이나 '실패까지의 성공 시간'으로 채우지 않았다.
새로 실행한 것은 24개 설정의 결정 180개와 panel별 구간 540개의 계산이다. 원래의 168개 로그 파일을 고정해 보관했지만 핵심 대비에는 예산별 24개 경로, 총 48개 기록이 쓰인다. 모두 같은 12개 저자 설계 템플릿에서 나온다. 540개를 독립 표본으로 계산하지 않는다. 새로운 모델 API 요청·worker 실행·실제 state audit·자연 실패 이력·독립 단위·fit은 모두 0이다. 조회 숫자는 이미 저장된 점수의 검사량이다.

23. 중요한 비교 논문을 어디까지 적용했는가

Golovin–Krause의 adaptive submodularity, 수정 v5 (https://arxiv.org/html/1003.3967v5)는 관측된 선택 결과에 따라 다음 항목을 고르는 정책을 다룬다. 우리도 관측 정보의 경계를 명시하지만, 그 논문의 prior나 조건부 기대 이득 계산을 구현한 것은 아니다. 특히 우리 최종 중요도는 미래 commit에 따라 달라지므로 필요한 구조 조건을 증명하지 않은 상태에서 greedy 근사 보장을 가져오면 안 된다. v5의 proof 수정 공지도 확인했으며, 오래된 최소 비용 coverage 보장을 무조건 인용하지 않는다.
COCO의 공식 성능 평가 문서 (https://numbbo.github.io/coco-doc/perf-assessment/)를 실제 다시 읽고, 고정 예산에서 도달한 값과 목표까지 필요한 횟수를 구분하는 보고 원칙을 참고했다. COCO 함수나 optimizer를 실행한 것은 아니다. 논문에 실린 외부 성능 수치를 이번 재생 표에 섞지 않았다. 조회 cap별 폭, 첫 증명 시점, 실제 읽은 캐시 수, 아직 미도달인 표적을 서로 다른 필드로 보존한다.

24. 별도 검산과 다음에 받아야 할 실제 증거

실행 전에 PLAN·분석 코드·검산 코드·168개 입력을 새 실행 폴더에 고정했다. 검산 코드는 저자가 별도로 작성했으며 분석 함수를 import하지 않는다. 비용 1짜리 구간을 직접 합쳐 점유 계수를 다시 계산하고, 180개 결정을 재현하며, 540개 유리수 bound를 확인한다. 미래 도착 제약을 만족하는 부분집합 후보 744개를 열거해 동적 계획법의 8개 하한과 일치함을 확인했다. 이 검산도 같은 저자의 계산이며 독립 심사자의 검토와 구분한다.
다음 단계는 새로운 commit stream을 보기 전에 selector를 고정한 실제 전향 실험이다. 매 시점 받은 view와 선택을 저장하고, 원래 worker 호출·실제 감사 호출·캐시 적중·오류·경과 시간을 따로 기록해야 한다. 데이터 수집 중 미래 commit이나 최종 계수가 selector에 들어오면 해당 운영 비용 주장은 중단한다. 같은 완전 상태·같은 입력의 반복 실행 값이 달라지면 결정론적 재사용 가정도 재검토한다. 현재는 실행 가능한 정보 계약과 유한 재생 비교를 얻었고, 현장 절약이나 자연 실패에서의 우월성은 아직 얻지 않았다.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.